理解 FlashAttention

Understanding FlashAttention

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Memory Traffic	内存流量	
FlashAttention	FlashAttention（方法名保留）	
Online Softmax	在线 Softmax	
IO Complexity	输入输出复杂度	
Attention	注意力机制	Attention
Scaled Dot-product Attention	缩放点积注意力	SDPA
High-bandwidth Memory	高带宽内存	HBM
GPU Memory Hierarchy	GPU 内存层次结构	
Dense Attention	稠密注意力	
Self-attention	自注意力	
Attention Head	注意力头	
Floating-point	浮点	
GPU Generation	GPU 代际	
Additive Mask	加性掩码	
Shared Memory	共享内存	
Score Matrix	分数矩阵	
FLOP Count	浮点运算次数	
Registers	寄存器	
Rounding	舍入	
Softmax	Softmax 归一化	Softmax
Kernels	内核	
Values	值	
Query	查询	Q
Core	核心	
Key	键	K
Exact Attention	精确注意力	
Blockwise Merge	分块合并	
Backward Pass	反向传播	
Forward Pass	前向传播	
Queries	查询	
Tiling	分块	
FLOPs	浮点运算次数	FLOPs

英文原词	中文译名	缩写／符号
Keys	键	
Dot-product Attention	点积注意力	
Work Partitioning	工作划分	
Linear Attention	线性注意力	
Bitwise Identity	逐位一致性	
Local Attention	局部注意力	
PagedAttention	PagedAttention（方法名保留）	
Data Movement	数据移动	
Mental Model	理解框架	
Performance	性能	
Generation	代际	
Precision	精度	
Training	训练	
Scaling	缩放	
Prefill	预填充	
Dropout	随机失活	
Matmul	矩阵乘法	
GEMM	通用矩阵乘法	GEMM
Row-wise Softmax	逐行 Softmax	
Matrix	矩阵	
Kernel	内核	
Model	模型	
Matrix Multiplication	矩阵乘法	
Recomputation	重计算	
Fused Kernels	融合内核	
Throughput	吞吐量	
Operator	算子	
Head Dimension	头维度	
Device Memory	设备内存	
Memory-bound	内存受限	
Resident	驻留	
Tensor	张量	
Models	模型	
Mask	掩码	

英文原词	中文译名	缩写／符号
Attention Heads	注意力头	
Occupancy	占用率	
Tokens	词元	
Efficient Attention	高效注意力	
Memory Hierarchy	内存层次结构	
Normalization	归一化	
Value	值	V
Accumulator	累加器	
Matrices	矩阵	
Vector	向量	
Vectors	向量	
Accumulation	累加	
Batching	批处理	
Warp	线程束	
Floating Point	浮点	
Memory Footprint	内存占用	
Transformer	Transformer 模型	Transformer
Activation	激活	
Serving	推理服务	
Causal Self-attention	因果自注意力	
Gradient	梯度	
Tensor-core Throughput	张量核心吞吐量	
Synchronization	同步	
Kernel Launch	内核启动	
Tensor Cores	张量核心	
Latency	延迟	
Key/Value Heads	键值头	KV Head
Query Heads	查询头	
Query Head	查询头	
KV Heads	键值头	
Heads	头	
Causal Attention	因果注意力	
Inference	推理	
KV Cache	键值缓存	KV Cache

英文原词	中文译名	缩写／符号
Thread Block	线程块	
Benchmark	基准测试	
Pipeline	流水线	
Warps	线程束	
Batch Size	批大小	
FLOP	浮点运算	
Quantization	量化	
Accuracy	准确性	
Tensor Memory Accelerator	张量内存加速器	TMA
Quantization Error	量化误差	
Baseline	基线	
Memory Bandwidth	内存带宽	
MMA	矩阵乘加	MMA
Evidence	证据	
GPU Utilization	GPU 利用率	
Fused Kernel	融合内核	
Fragmentation	碎片化	
Local Window	局部窗口	
Cache Capacity	缓存容量	
Token	词元	
Boolean Mask	布尔掩码	
Profiling	性能剖析	
Large Language Model	大语言模型	LLM
Head Dimensions	头维度	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

了解 FlashAttention · 第 8 页 ［原文 1］

如何阅读本手册 · 第 10 页 ［原文 2］

内容 · 第 12 页 ［原文 3］

内容续 · 第 14 页 ［原文 4］

1 FlashAttention实际优化了什么 · 第 16 页 ［原文 5］

2 注意力机制（Attention） 方程不是实现 · 第 18 页 ［原文 6］

3 GPU 内存层次结构（GPU Memory Hierarchy） 以及为什么 IO 很重要 · 第 20 页 ［原文 7］

4 为什么实现 S 和 P 成本昂贵 · 第 22 页 ［原文 8］

5 稠密算术仍然是二次的 · 第 24 页 ［原文 9］

6 内存效率高的 精确注意力（Exact Attention） 早于 FlashAttention · 第 26 页 ［原文 10］

7 分块（Tiling） 查询（Queries）、键（Keys） 和 值（Values） · 第 28 页 ［原文 11］

8 Softmax 归一化（Softmax） 是流式传输的难点 · 第 30 页 ［原文 12］

9 在线 Softmax（Online Softmax） 从第一原理 · 第 32 页 ［原文 13］

10 分块合并（Blockwise Merge） 重复 · 第 34 页 ［原文 14］

11 一个完整的数值例子 · 第 36 页 ［原文 15］

12 FlashAttention 前向传播（Forward Pass） · 第 38 页 ［原文 16］

13 为什么密集FlashAttention是准确的 · 第 40 页 ［原文 17］

14 输入输出复杂度（IO Complexity）：定理实际上说了什么 · 第 42 页 ［原文 18］

15 内存复杂度：线性辅助状态，非线性计算 · 第 44 页 ［原文 19］

16 因果掩蔽和瓦片跳跃 · 第 46 页 ［原文 20］

17 反向传播（Backward Pass）：重新计算而不是存储 · 第 48 页 ［原文 21］

18 为什么更多的浮点运算次数（FLOPs）仍然可以更快 · 第 50 页 ［原文 22］

19 MHA、MQA 和 GQA 兼容性 · 第 52 页 ［原文 23］

20 个可变长度、局部注意力（Local Attention） 和 随机失活（Dropout） · 第 54 页 ［原文 24］

21 FlashAttention-2：发生了什么变化 · 第 56 页 ［原文 25］

22 FA2 跨序列图块的并行性 · 第 58 页 ［原文 26］

23 FA2 工作划分（Work Partitioning） 和非 矩阵乘法（Matmul） 浮点运算次数（FLOPs） · 第 60 页 ［原文 27］

24 FlashAttention-3：料斗 代际（Generation） · 第 62 页 ［原文 28］

25 FA3 异步：重叠 数据移动（Data Movement）、通用矩阵乘法（GEMM） 和 Softmax 归一化（Softmax） · 第 64 页 ［原文 29］

26 FA3 FP8：性能（Performance） 不假装 精度（Precision） 是免费的 · 第 66 页 ［原文 30］

27 FlashAttention-4：布莱克韦尔 代际（Generation） · 第 68 页 ［原文 31］

28 FA4 和非对称硬件 缩放（Scaling）· 第 70 页 ［原文 32］
29 FA4实施及现状 · 第 72 页 ［原文 33］
30 FlashAttention-1 到 -4 比较 · 第 74 页 ［原文 34］
31 PyTorch 今天缩放-点积注意力（Dot-product Attention）· 第 76 页 ［原文 35］
32 精确性不是 逐位一致性（Bitwise Identity）· 第 78 页 ［原文 36］
33 FlashAttention 与 PagedAttention · 第 80 页 ［原文 37］
34 FlashAttention 与稀疏和 线性注意力（Linear Attention）· 第 82 页 ［原文 38］
35 训练（Training）、预填充（Prefill） 和解码是不同的机制 · 第 84 页 ［原文 39］
36 种常见的实施错误 · 第 86 页 ［原文 40］
37 常见误解和实用 理解框架（Mental Model）· 第 88 页 ［原文 41］
参考文献和进一步阅读 · 第 90 页 ［原文 42］
参考图：针对哪个问题阅读什么内容 · 第 93 页 ［原文 43］

了解 FlashAttention

对应原文第 1 页；点击查看相邻原文对照页

了解 FlashAttention IO 感知型 注意力机制（Attention） 如何在不近似 注意力机制（Attention） 的情况下使 Transformers 更快 精确平铺 注意力机制（Attention） 的技术手册：GPU 内存流量（Memory Traffic）、在线 Softmax（Online Softmax）、前向和后向传递、输入输出复杂度（IO Complexity）、从 FlashAttention-1 到 FlashAttention-4 以及当前框架行为。

Understanding FlashAttention

How IO-Aware Attention Makes Transformers Faster Without Approximating Attention

---

A technical handbook on exact tiled attention: GPU memory traffic, online softmax, forward and backward passes, IO complexity, the evolution from FlashAttention-1 through FlashAttention-4, and current framework behavior.

@techNmak

如何阅读本手册

对应原文第 2 页；点击查看相邻原文对照页

当三个不同的问题分开时，如何阅读本手册 FlashAttention 是最容易理解的：

1. 正在计算什么数学函数？对于 稠密注意力（Dense Attention），目标仍然是普通的 缩放点积注意力（Scaled Dot-product Attention）。

2. 该函数需要多少算术运算？密集全对 查询（Query）-键（Key） 评分在序列长度上保持二次方。

3. 实现如何通过 GPU 内存层次结构（GPU Memory Hierarchy） 移动数据？这就是 FlashAttention 极大改变算法执行的地方。[4]

核心教训是实际运行速度不仅仅由 浮点运算次数（FLOP Count） 决定。算法基本上可以执行相同的数学工作，有时甚至重新计算中间 值（Values），但运行速度更快，因为它往返于 高带宽内存（High-bandwidth Memory） 的数据要少得多。

核心（Core） 区别。密集 FlashAttention 是精确的 注意力机制（Attention） 算法：它不会用低秩、稀疏、核化或近似公式替换 Softmax 归一化（Softmax） 注意力机制（Attention）。“精确”是指数学注意力机制（Attention） 计算。 浮点（Floating-point） 内核（Kernels） 仍然可能因小的 舍入（Rounding） 效果而有所不同，因为操作被重新排序。[4,10,13]

符号 对于一个 注意力头（Attention Head），令

对于自注意力（Self-attention），通常Nq = Nk = N。缩放后的 分数矩阵（Score Matrix） 为

其中 B 代表可选的 加性掩码（Additive Mask） 或偏差，并且

我们将 HBM 用于大型片外高带宽 GPU 内存和片上内存，作为更小、更快的存储（例如 寄存器（Registers） 和 共享内存（Shared Memory）/SRAM）的广泛教学术语。确切的硬件详细信息因 GPU 代际（GPU Generation） 而异。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

How to read this handbook

FlashAttention is easiest to understand when three different questions are kept separate:

1. **What mathematical function is being computed?** For dense attention, the target remains ordinary scaled dot-product attention.
2. **How much arithmetic does that function require?** Dense all-pairs query-key scoring remains quadratic in sequence length.
3. **How does the implementation move data through the GPU memory hierarchy?** This is where FlashAttention changes the algorithmic execution dramatically.[4]

The central lesson is that wall-clock speed is not determined by FLOP count alone. An algorithm can perform essentially the same mathematical work, and sometimes even recompute intermediate values, yet run faster because it moves far less data to and from high-bandwidth memory.

> **Core distinction.** Dense FlashAttention is an *exact attention algorithm*: it does not replace softmax attention with a low-rank, sparse, kernelized, or approximate formula. "Exact" refers to the mathematical attention computation. Floating-point kernels can still differ by small rounding effects because operations are reordered.[4, 10, 13]

Notation

For one attention head, let

$$Q \in \mathbb{R}^{N_q \times d}, \qquad K \in \mathbb{R}^{N_k \times d}, \qquad V \in \mathbb{R}^{N_k \times d_v}.$$

For self-attention, typically $N_q = N_k = N$. The scaled score matrix is

$$S = \frac{QK^\top}{\sqrt{d}} + B,$$

where B represents an optional additive mask or bias, and

$$P = \mathrm{softmax}_{\mathrm{row}}(S), \qquad O = PV.$$

We use **HBM** for large off-chip high-bandwidth GPU memory and **on-chip memory** as a broad teaching term for much smaller, faster storage such as registers and shared memory/SRAM. Exact hardware details vary by GPU generation.

内容

对应原文第 3 页；点击查看相邻原文对照页

目录 1 FlashAttention 实际优化了什么 5 2 注意力机制（Attention） 方程不是实现 6 3 GPU 内存层次结构（GPU Memory Hierarchy） 以及为什么 IO 很重要 7 4 为什么具体化 S 和 P 成本很高 8 5 稠密算术仍然是二次的 9 6 内存效率 精确注意力（Exact Attention） 早于 FlashAttention 10 7 分块（Tiling） 查询（Queries）、键（Keys） 和 值（Values） 11 8 Softmax 归一化（Softmax） 是流式传输的困难部分 12 9 在线 Softmax（Online Softmax） 第一原理 13 10 分块合并（Blockwise Merge） 递归 14 11 完整的数值示例 15 12 FlashAttention 前向传播（Forward Pass） 16 13 为什么稠密 FlashAttention 是精确的 17 14 输入输出复杂度（IO Complexity）：定理实际上说了什么 18 15 内存复杂性：线性辅助状态，不是线性计算 19 16 因果屏蔽和切片跳过 20 17 反向传播（Backward Pass）：重新计算而不是存储21 18 为什么更多的 浮点运算次数（FLOPs） 仍然可以更快 22 19 MHA、MQA 和 GQA 兼容性 23

Contents

1	What FlashAttention actually optimizes	5
2	The attention equation is not the implementation	6
3	GPU memory hierarchy and why IO matters	7
4	Why materializing S and P is expensive	8
5	Dense arithmetic is still quadratic	9
6	Memory-efficient exact attention predates FlashAttention	10
7	Tiling queries, keys, and values	11
8	Softmax is the difficult part of streaming	12
9	Online softmax from first principles	13
10	The blockwise merge recurrence	14
11	A complete numerical example	15
12	FlashAttention forward pass	16
13	Why dense FlashAttention is exact	17
14	IO complexity: what the theorem actually says	18
15	Memory complexity: linear auxiliary state, not linear compute	19
16	Causal masking and tile skipping	20
17	Backward pass: recompute instead of store	21
18	Why more FLOPs can still be faster	22
19	MHA, MQA, and GQA compatibility	23

内容续

对应原文第 4 页；点击查看相邻原文对照页

内容续 20 可变长度、局部注意力（Local Attention） 和 随机失活（Dropout） 24 21 FlashAttention-2：发生了什么变化 25 22 跨序列图块的 FA2 并行性 26 23 FA2 工作划分（Work Partitioning） 和非 矩阵乘法（Matmul） 浮点运算次数（FLOPs） 27 24 FlashAttention-3：料斗 代际（Generation） 28 25 FA3 异步：重叠 数据移动（Data Movement）、通用矩阵乘法（GEMM） 和 Softmax 归一化（Softmax） 29 26 FA3 FP8：性能（Performance），无需假装 精度（Precision） 空闲 30 27 FlashAttention-4：Blackwell 代际（Generation） 31 28 FA4 和非对称硬件 缩放（Scaling） 32 29 FA4 实现和当前状态 33 30 FlashAttention-1 到 -4 比较 34 31 今天 PyTorch 缩放的 点积注意力（Dot-product Attention） 35 32 准确性是不是 逐位一致性（Bitwise Identity） 36 33 FlashAttention 与 PagedAttention 37 34 FlashAttention 与稀疏和 线性注意力（Linear Attention） 38 35 训练（Training）、预填充（Prefill） 和解码是不同的机制 39 36 常见实现错误 40 37常见误解和实用 理解框架（Mental Model） 41

参考资料和进一步阅读 42 FlashAttention 系列随着 GPU 硬件的发展而发展。第 1-20 章解释了持久的算法思想。第 21-31 章隔离了 代际（Generation） 特定和框架特定的细节，以便 核心（Core） 理解框架（Mental Model） 不依赖于一个 CUDA 版本或一个 GPU 代际（GPU Generation）。

Contents continued

20	Variable lengths, local attention, and dropout	24
21	FlashAttention-2: what changed	25
22	FA2 parallelism across sequence tiles	26
23	FA2 work partitioning and non-matmul FLOPs	27
24	FlashAttention-3: the Hopper generation	28
25	FA3 asynchrony: overlap data movement, GEMM, and softmax	29
26	FA3 FP8: performance without pretending precision is free	30
27	FlashAttention-4: the Blackwell generation	31
28	FA4 and asymmetric hardware scaling	32
29	FA4 implementation and current status	33
30	FlashAttention-1 through -4 compared	34
31	PyTorch scaled-dot-product attention today	35
32	Exactness is not bitwise identity	36
33	FlashAttention vs. PagedAttention	37
34	FlashAttention vs. sparse and linear attention	38
35	Training, prefill, and decode are different regimes	39
36	Common implementation mistakes	40
37	Common misconceptions and the practical mental model	41
	References and further reading	42

The FlashAttention family evolves with GPU hardware. Chapters 1–20 explain the durable algorithmic ideas. Chapters 21–31 isolate generation-specific and framework-specific details so that the core mental model does not depend on one CUDA release or one GPU generation.

1 FlashAttention实际优化了什么

对应原文第 5 页；点击查看相邻原文对照页

1 FlashAttention实际优化了什么 从普通的注意力机制（Attention）函数开始：

教科书的实现通常使它看起来像三个大型运算：形成分数矩阵S，应用逐行 Softmax（Row-wise Softmax）获得P，然后将P乘以V。从数学上来说，这很好。然而，在 GPU 上，将巨大的中间 矩阵（Matrix） 写入 HBM 并将其读回的成本可能比公式所示的要高得多。

FlashAttention 的核心贡献是使算法具有 IO 感知能力。它将计算划分为适合快速片上内存的块，流式传输 K 和 V 块，并维护足够的 逐行 Softmax（Row-wise Softmax） 状态以产生精确的输出，而无需在 HBM 中实现完整的 N×N 注意力机制（Attention） 矩阵（Matrix）。 [4]

更改内容：执行计划、内存流量（Memory Traffic） 和存储的中间体。

不变的是：正在评估密集缩放的 点积注意力（Dot-product Attention） 函数。

这种区别就是为什么“FlashAttention 是一种更快的 注意力机制（Attention）”可能会产生误导。最好将其视为一种算法和 内核（Kernel） 系列，用于在加速器上有效评估 注意力机制（Attention）。 模型（Model） 可以使用因果屏蔽、RoPE、MQA/GQA 或其他 注意力机制（Attention） 功能，并且仍然在下面使用 FlashAttention 实现。

原始论文将该方法与通过改变数学问题来减少算术的近似 注意力机制（Attention） 方法进行了对比。Dense FlashAttention 不会进行该交易。同一篇论文还介绍了块稀疏扩展，但稀疏扩展是一种不同的情况，因为省略块会改变计算交互的方式。 [4]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

1 What FlashAttention actually optimizes

Start with the ordinary attention function:

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B\right) V.$$

A textbook implementation often makes this look like three large operations: form the score matrix S, apply row-wise softmax to obtain P, then multiply P by V. Mathematically that is fine. On a GPU, however, writing a huge intermediate matrix to HBM and reading it back can be far more expensive than the equation suggests.

FlashAttention's central contribution is to make the algorithm **IO-aware**. It partitions the computation into tiles that fit in fast on-chip memory, streams blocks of K and V, and maintains enough row-wise softmax state to produce the exact output without materializing the full $N \times N$ attention matrix in HBM.[4]

> **What changes:** the execution schedule, memory traffic, and stored intermediates.
> **What does not change:** the dense scaled-dot-product attention function being evaluated.

This distinction is why "FlashAttention is a faster kind of attention" can be misleading. It is better thought of as an algorithm and kernel family for evaluating attention efficiently on accelerators. A model can use causal masking, RoPE, MQA/GQA, or other attention features and still use a FlashAttention implementation underneath.

The original paper contrasts this approach with approximate attention methods that reduce arithmetic by changing the mathematical problem. Dense FlashAttention does not make that trade. The same paper also introduced a *block-sparse* extension, but that sparse extension is a different case because omitting blocks changes which interactions are computed.[4]

2 注意力机制（Attention） 方程不是实现

对应原文第 6 页；点击查看相邻原文对照页

2 注意力机制（Attention） 方程不是实现 两种实现可以计算相同的数学函数，同时移动不同数量的数据。

一个简单的物化实现在概念上可以做到：

该演示隐藏了张量所在的位置。如果在第一个 矩阵乘法（Matrix Multiplication） 之后将 S 写入 HBM，读取 Softmax 归一化（Softmax），写回 P，然后再次读取 P 以进行 PV 乘法，则 GPU 将花费大量时间在内存中移动 N 2 对象。

FlashAttention 重新组织相同的依赖关系。 Q、K 和 V 块被带到计算单元附近，分数图块在本地生成和消耗，并且只有紧凑的行统计数据加上输出需要在图块之间保留。[4, 5]

算法课。计算图不是内存调度。在纸上写 P = Softmax 归一化（Softmax）(QK⊤) 不需要实现将所有 QK⊤ 或 P 立即存储在片外存储器中。

这个想法可以推广到 注意力机制（Attention） 之外。 融合内核（Fused Kernels）、分块（Tiling）、重计算（Recomputation） 和 算子（Operator） 调度通常会用少量的额外算术来换取更少的大型中间体移动。在现代加速器上，这可能是正确的交易，因为 矩阵（Matrix） 乘法 吞吐量（Throughput） 的增长速度比内存和执行层次结构的许多其他部分快得多。 FlashAttention-3 和 -4 使硬件依赖性变得越来越明确。[6, 7]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

2 The attention equation is not the implementation

Two implementations can compute the same mathematical function while moving very different amounts of data.

A simple materializing implementation may conceptually do:

$$S \leftarrow QK^\top/\sqrt{d}, \qquad P \leftarrow \mathrm{softmax}(S), \qquad O \leftarrow PV.$$

This presentation hides where tensors live. If S is written to HBM after the first matrix multiplication, read for softmax, P is written back, and then P is read again for the PV multiplication, the GPU spends significant time moving an N^2 object through memory.

FlashAttention reorganizes the same dependencies. Blocks of Q, K, and V are brought near the compute units, score tiles are produced and consumed locally, and only compact row-wise statistics plus the output need to persist across tiles.[4, 5]

> **Algorithmic lesson.** A computational graph is not a memory schedule. Writing $P = \mathrm{softmax}(QK^\top)$ on paper does not require an implementation to store all of $QK^\top$ or P in off-chip memory at once.

This idea generalizes beyond attention. Fused kernels, tiling, recomputation, and operator scheduling often trade a small amount of extra arithmetic for much less movement of large intermediates. On modern accelerators, that can be the right trade because matrix-multiply throughput has grown much faster than many other parts of the memory and execution hierarchy. FlashAttention-3 and -4 make that hardware dependence increasingly explicit.[6, 7]

3 GPU 内存层次结构（GPU Memory Hierarchy） 以及为什么 IO 很重要

对应原文第 7 页；点击查看相邻原文对照页

3 GPU 内存层次结构（GPU Memory Hierarchy） 以及为什么 IO 很重要 GPU 暴露的是一个层次结构，而不是一个同样快速的统一内存池。确切的名称和容量因架构而异，但有用的 理解框架（Mental Model）是：

寄存器（Registers） / 非常本地状态 片上 共享内存（Shared Memory） / SRAM HBM / 设备内存（Device Memory） 负载块馈送计算 HBM 很大，但数据必须传输到计算单元。片上存储要小得多，但重复使用要便宜得多。原始的 FlashAttention 使用 HBM 和 SRAM 分析 模型（Models） 这种不对称性，并明确优化它们之间的传输数量。 [4]

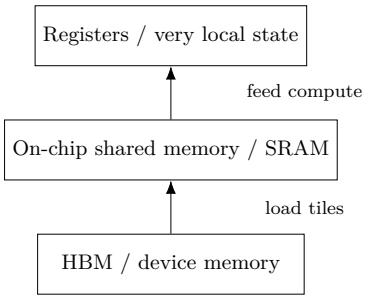
分块（Tiling） 的目的不仅仅是“让张量更小”。芯片上的tile为驻留（Resident）时，是为了增加复用。 Q 块在其部分 Softmax 归一化（Softmax）/输出状态被写回之前可以与多个 K/V 块交互。相反，K/V 块可以根据所选的时间表通过 查询（Query） 块进行流式传输。

当数据的放置和移动成为算法的一部分时，内核（Kernel） 就会成为 IO 感知的，而不是事后才想到一系列单独启动的 张量（Tensor） 操作。

不要将其变成“注意力机制（Attention） 始终是“内存受限（Memory-bound）”的通用口号。瓶颈取决于序列长度、头维度（Head Dimension）、dtype、掩码（Mask） 模式、GPU 代际（GPU Generation）、向前与向后以及 内核（Kernel） 正在运行。 FA3 和 FA4 存在的部分原因是，随着硬件的变化，主要瓶颈也发生了变化。[6, 7]

3 GPU memory hierarchy and why IO matters

GPUs expose a hierarchy rather than one uniform pool of equally fast memory. The exact names and capacities vary by architecture, but a useful mental model is:



HBM is large, but data must travel to the compute units. On-chip storage is much smaller, but reuse there is much cheaper. The original FlashAttention analysis models this asymmetry using HBM and SRAM and explicitly optimizes the number of transfers between them.[4]

The purpose of tiling is not simply "make tensors smaller." It is to increase **reuse while a tile is resident on chip**. A block of Q can interact with multiple K/V blocks before its partial softmax/output state is written back. Conversely, K/V blocks can be streamed through query blocks according to the chosen schedule.

> A kernel becomes IO-aware when the placement and movement of data are part of the algorithm, rather than an afterthought left to a sequence of separately launched tensor operations.

Do not turn this into a universal slogan that attention is always "memory-bound." The bottleneck depends on sequence lengths, head dimension, dtype, mask pattern, GPU generation, forward vs. backward, and which kernel is running. FA3 and FA4 exist partly because, as hardware changed, the dominant bottlenecks changed too.[6, 7]

4 为什么实现 S 和 P 成本昂贵

对应原文第 8 页；点击查看相邻原文对照页

4 为什么具体化 S 和 P 很昂贵 二次中间体很快就会变得具体。假设一个批次包含一个序列，其中 32 注意力头（Attention Heads），序列长度 N = 8192，以及两字节 dtype，例如 FP16 或 BF16。一颗密张量（Tensor）有形状

包含 32×81922 个元素。每个元素两个字节，即

对于本例来说，这是一个满分或类似 张量（Tensor） 的概率的大小。

简单的分解可能会在多个阶段产生这种规模的数组。这并不意味着每个现代框架都使 S 和 P 同时保持活动状态，并且编译器融合已经可以避免一些流量。

该示例说明了基本问题：denseN 2 中间体足够大，重复写入和重新读取它可以控制内存使用和带宽。

FlashAttention 避免将完整的 矩阵（Matrix） 存储在 HBM 中。它形成得分图块，在图块位于芯片上时应用 Softmax 归一化（Softmax） 更新，立即使用这些概率累积来自 V 的相应贡献，然后丢弃该图块。 [4]

重要措辞：FlashAttention 无需将完整的 注意力机制（Attention） 矩阵（Matrix） 实现为片外中间体。它不会删除 稠密注意力（Dense Attention） 所需的逻辑成对交互。

这也是为什么内存优势在 训练（Training） 期间特别重要，否则天真的 autograd 会需要 反向传播（Backward Pass） 的大型中间体。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

4 Why materializing S and P is expensive

The quadratic intermediate becomes concrete very quickly. Suppose a batch contains one sequence, with 32 attention heads, sequence length $N = 8192$, and a two-byte dtype such as FP16 or BF16. One dense tensor with shape

$$[1, 32, 8192, 8192]$$

contains 32×8192^2 elements. At two bytes per element, that is

$$32 \times 8192^2 \times 2 = 4,294,967,296 \text{ bytes} = 4 \text{ GiB}.$$

That is the size of *one* full score- or probability-like tensor for this example.

A naïve decomposition may produce arrays of this scale at multiple stages. This does not mean every modern framework keeps both S and P alive simultaneously, and compiler fusion can already avoid some traffic. The example illustrates the fundamental problem: a dense N^2 intermediate is large enough that repeatedly writing and rereading it can dominate memory use and bandwidth.

FlashAttention avoids storing the full matrix in HBM. It forms score tiles, applies the softmax update while the tile is on chip, immediately uses those probabilities to accumulate the corresponding contribution from V, then discards the tile.[4]

> **Important wording:** FlashAttention removes the need to materialize the full attention matrix as an off-chip intermediate. It does *not* remove the logical pairwise interactions required by dense attention.

This is also why the memory benefit is especially important during training, where naïve autograd would otherwise want large intermediates for the backward pass.

5 稠密算术仍然是二次的

对应原文第 9 页；点击查看相邻原文对照页

5 稠密算术仍然是二次的 对于 自注意力（Self-attention） 和 N 词元（Tokens） 且头尺寸确定，形成所有 查询（Query）-键（Key） 分数需要成比例的工作

将概率乘以 值（Values） 会添加另一个具有相同宽阶的密集成对 矩阵乘法（Matrix Multiplication）。 FlashAttention 重新组织了这些操作，但它并没有停止评估 查询（Query）-键（Key） 交互的密集集。 [4, 5]

所以这三个陈述一定不能混淆：

- 算术复杂度：稠密注意力（Dense Attention） 仍为O(N 2d)。

- 大型中间存储：FlashAttention 避免了 HBM 中的 O(N 2) 物化分数/概率 张量（Tensor）。

- HBM 流量：原论文证明其两级内存 模型（Model） 下的 IO 成本比标准物化 注意力机制（Attention） 更低。[4]

这解释了一种方法如何能够在不使长上下文“自由”的情况下使更长的序列变得实用。 N 加倍仍然使密集 查询（Query）-键（Key） 对的数量大致增加四倍。 FlashAttention 主要攻击该计算的数据移动和内存占用方面。

如果要减少 查询（Query）-键（Key） 对本身的数量，则需要不同的数学结构：例如稀疏性、局部模式或不同的 注意力机制（Attention） 公式。这些选择可以更改 模型（Model） 行为，并且在概念上与密集 FlashAttention 分开。

报告加速时，始终将渐近算术与测量的运行时间区分开来。在相同的 O(N 2d) 复杂度下，内核（Kernel） 可以变得快几倍，因为常数因子、占用率（Occupancy）、融合和 内存流量（Memory Traffic） 发生巨大变化。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

5 Dense arithmetic is still quadratic

For self-attention with N tokens and head dimension d, forming all query-key scores requires work proportional to

$$N^2 d.$$

Multiplying the probabilities by values adds another dense pairwise matrix multiplication of the same broad order. FlashAttention reorganizes these operations, but it does not stop evaluating the dense set of query-key interactions.[4, 5]

So three statements must not be confused:

- **Arithmetic complexity:** dense attention remains $O(N^2 d)$.
- **Large intermediate storage:** FlashAttention avoids an $O(N^2)$ materialized score/probability tensor in HBM.
- **HBM traffic:** the original paper proves a lower IO cost under its two-level memory model than standard materializing attention.[4]

This explains how a method can make much longer sequences practical without making long context "free." Doubling N still roughly quadruples the number of dense query-key pairs. FlashAttention mainly attacks the data-movement and memory-footprint side of that computation.

> If you want to reduce the number of query-key pairs themselves, you need a different mathematical structure: for example sparsity, a local pattern, or a different attention formulation. Those choices can change model behavior and are conceptually separate from dense FlashAttention.

When reporting speedups, always distinguish asymptotic arithmetic from measured runtime. A kernel can become several times faster at the same $O(N^2 d)$ complexity because the constant factors, occupancy, fusion, and memory traffic change dramatically.

6 内存效率高的 精确注意力（Exact Attention） 早于 FlashAttention

对应原文第 10 页；点击查看相邻原文对照页

6 内存高效的 精确注意力（Exact Attention） 早于 FlashAttention 从历史上看，说 FlashAttention 首先发现精确注意力（Exact Attention） 可以避免二次内存是不准确的。

Rabe 和 Staats 在 FlashAttention 之前表明，注意力机制（Attention） 不需要就序列长度而言需要 O(N 2) 内存。他们的工作提供了精确的内存高效算法，同时保留二次时间，以及具有次二次内存的实用加速器实现。 [3]

FlashAttention 的独特贡献是将内存效率转变为显式 IO 感知 GPU 算法：围绕加速器 内存层次结构（ Memory Hierarchy） 平铺计算，分析 HBM 访问，融合相关操作，并显示大量的实际运行增益。 [4]

在线 Softmax（Online Softmax） 也有更早的谱系。 Milakov 和 Gimelshein 描述了一种在线递归，可以用更少的内存访问来计算经典的稳定 Softmax 归一化（Softmax） 标准化器。 FlashAttention 将相同类型的 running-max/running-normalizer 想法构建到平铺的 注意力机制（Attention） 中，同时还累积 值（Value） 加权输出。 [2, 4]

因此，精确的血统是：

- 稳定/在线 Softmax（Online Softmax） 提供流式 归一化（Normalization） 工具，

- 较早的内存-高效注意力（Efficient Attention） 显示精确注意力（Exact Attention） 不需要存储N 2 个状态 ，

- FlashAttention 共同设计了 分块（Tiling）、Softmax 归一化（Softmax） 状态和 GPU IO，以使该方法在实践中快速运行。

这种区别很重要，因为“内存效率”和“硬件 模型（Model） IO 最佳”并不是同一说法。

6 Memory-efficient exact attention predates FlashAttention

It is historically inaccurate to say FlashAttention first discovered that exact attention can avoid quadratic memory.

Rabe and Staats showed before FlashAttention that attention need not require $O(N^2)$ memory with respect to sequence length. Their work gave exact memory-efficient algorithms while retaining quadratic time, and a practical accelerator implementation with subquadratic memory.[3]

FlashAttention's distinct contribution was to turn memory efficiency into an explicit **IO-aware GPU algorithm**: tile the computation around the accelerator memory hierarchy, analyze HBM accesses, fuse the relevant operations, and show substantial wall-clock gains.[4]

Online softmax also has an earlier lineage. Milakov and Gimelshein described an online recurrence that computes the classical stable softmax normalizer with fewer memory accesses. FlashAttention builds the same kind of running-max/running-normalizer idea into tiled attention, while also accumulating the value-weighted output.[2, 4]

A precise lineage is therefore:

- stable / online softmax provides a streaming normalization tool,
- earlier memory-efficient attention shows exact attention need not store N^2 state,
- FlashAttention co-designs tiling, softmax state, and GPU IO to make the approach fast in practice.

This distinction matters because "memory efficient" and "IO optimal for a hardware model" are not the same claim.

7 分块（Tiling） 查询（Queries）、键（Keys） 和 值（Values）

对应原文第 11 页；点击查看相邻原文对照页

7 分块（Tiling） 查询（Queries）、键（Keys） 和 值（Values） 不是一次形成所有 NqNk 分数，而是将矩阵（Matrices） 划分为块。对于一个 查询（Query） 块Qi 和一个 键（Key）/值（Value） 块(Kj,V j)，计算

+毕。

分数tileSij足够小，可以在计算单元附近进行处理。它的贡献被折叠到运行行统计数据和输出 累加器（Accumulator） 中，然后可以丢弃该图块。

从概念上讲，对于每个 查询（Query） 块：

1.加载Q的图块，

2.流兼容K/V图块，

3. 计算一个分数图块，

4.更新逐行 Softmax（Row-wise Softmax）状态，

5.积累相应的V贡献，

6. 移动到下一个图块，而不写入 globalS 或 P 矩阵（Matrix）。

实际的内核（Kernels）根据片上容量、头维度（Head Dimension）、GPU 代际（GPU Generation）、因果结构和工作划分（Work Partitioning）选择图块形状和循环顺序。原始 FlashAttention IO 分析使用源自 SRAM 容量 M 的切片大小。[4]

仅靠 分块（Tiling） 是不够的。 矩阵乘法（Matrix Multiplication） 瓷砖自然组成，因为总和可以累积。Softmax 归一化（Softmax） 通过共享的最大值和分母将一行中的每个分数结合在一起，因此我们需要一种方法来合并块，而无需立即看到整行。

这就是下一章中的 键（Key） 数学技巧。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

7 Tiling queries, keys, and values

Instead of forming all $N_q N_k$ scores at once, partition the matrices into blocks. For one query block Q_i and one key/value block (K_j, V_j), compute

$$S_{ij} = \frac{Q_i K_j^\top}{\sqrt{d}} + B_{ij}.$$

The score tile S_{ij} is small enough to be processed near the compute units. Its contribution is folded into running row statistics and an output accumulator, then the tile can be discarded.

Conceptually, for each query block:

1. load a tile of Q,
2. stream compatible K/V tiles,
3. compute one score tile,
4. update row-wise softmax state,
5. accumulate the corresponding V contribution,
6. move to the next tile without writing a global S or P matrix.

Actual kernels choose tile shapes and loop order based on on-chip capacity, head dimension, GPU generation, causal structure, and work partitioning. The original FlashAttention IO analysis uses tile sizes derived from SRAM capacity M.[4]

> Tiling alone is not enough. Matrix multiplication tiles compose naturally because sums can be accumulated. Softmax couples every score in a row through a shared maximum and denominator, so we need a way to merge blocks without seeing the full row at once.

That is the key mathematical trick in the next chapters.

8 Softmax 归一化（Softmax） 是流式传输的难点

对应原文第 12 页；点击查看相邻原文对照页

8 Softmax 归一化（Softmax） 是流式传输的困难部分 对于一行x1,...,x N 而言数值稳定的 Softmax 归一化（Softmax） 使用 m = max

exj-m, p j = exj-m

bym 减法可防止大的正 logits 溢出。但这似乎产生了一个流问题：如果后面的块可能包含更大的最大值，那么如何标准化早期的块？

答案是保留足够的统计数据，以便在最大值发生变化时可以重新调整。假设早期元素之后的运行状态是(mold,ℓold)并且新块具有maximummb。定义 mnew = max(模具,m b)。

在旧的最大值下累积的每个贡献都可以通过乘以转换为新的参考值

α=emold-mnew。

新块是相对于 samemnew 进行评估的。

这不是一个近似值。它是恒等式 ex-moldemold-mnew =ex-mnew。

运行最大值是数值参考点的变化。当该参考发生变化时，先前累积的指数可以在实际算术中精确地重新缩放，而不是从头开始重新计算。

在线归一化器递归早于 FlashAttention，并为流式传输提供了数学基础

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

8 Softmax is the difficult part of streaming

A numerically stable softmax for one row $x_1, \ldots, x_N$ uses

$$m = \max_j x_j, \qquad \ell = \sum_j e^{x_j - m}, \qquad p_j = \frac{e^{x_j - m}}{\ell}.$$

The subtraction by m prevents overflow from large positive logits. But it seems to create a streaming problem: how can an early block be normalized if a later block may contain a larger maximum?

The answer is to retain sufficient statistics that can be **rescaled** when the maximum changes. Suppose the running state after earlier elements is $(m_{\text{old}}, \ell_{\text{old}})$ and a new block has maximum m_b. Define

$$m_{\text{new}} = \max(m_{\text{old}}, m_b).$$

Every contribution accumulated under the old maximum can be converted to the new reference by multiplying it by

$$\alpha = e^{m_{\text{old}} - m_{\text{new}}}.$$

The new block is evaluated relative to the same m_{new}.

This is not an approximation. It is the identity

$$e^{x - m_{\text{old}}} e^{m_{\text{old}} - m_{\text{new}}} = e^{x - m_{\text{new}}}.$$

> The running maximum is a change of numerical reference point. When that reference changes, previously accumulated exponentials can be rescaled exactly in real arithmetic rather than recomputed from scratch.

The online-normalizer recurrence predates FlashAttention and provides the mathematical basis for streaming stable softmax.[2]

9 在线 Softmax（Online Softmax） 从第一原理

对应原文第 13 页；点击查看相邻原文对照页

9 在线 Softmax（Online Softmax） 从第一原理开始 处理标量 logitsx1,x 2,... 一次一个。初始化

观察xj后，更新mj = max(mj-1,x j)，ℓj =ℓj-1emj-1-mj +exj-mj。

Milakov 和 Gimelshein 表明，这会产生相同稳定的 Softmax 归一化（Softmax） 标准化器，同时比传统的安全 Softmax 归一化（Softmax） 过程需要更少的输入传递。 [2]

对于 注意力机制（Attention），我们还需要加权 值（Value） 总和。引入非标准化的累加器（Accumulator）

exj-mvj。

当最大值从 m 变为 m′ 时，重新缩放 ℓ 和 a byem-m′。然后在新参考下添加新指数和 值（Value） 贡献。

最后，

对于注意力机制（Attention），xj不是预先存储的固定输入向量（Vector）。每个 logits 块都是根据 矩阵（Matrix） 产品QiK⊤ 的需求生成的

d 加上 掩码（Mask）/偏差项。在线循环让 内核（Kernel） 立即消耗该块。

同样的想法逐行、逐块地工作，这使得平铺的精确 Softmax 归一化（Softmax）-注意力机制（Attention）前向传播（Forward Pass） 成为可能。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

9 Online softmax from first principles

Process scalar logits $x_1, x_2, \ldots$ one at a time. Initialize

$$m_0 = -\infty, \qquad \ell_0 = 0.$$

After observing x_j, update

$$m_j = \max(m_{j-1}, x_j),$$
$$\ell_j = \ell_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}.$$

Milakov and Gimelshein show that this produces the same stable softmax normalizer while requiring fewer passes over the input than the conventional safe-softmax procedure.[2]

For attention, we also need the weighted value sum. Introduce an unnormalized accumulator

$$a = \sum_j e^{x_j - m} v_j.$$

When the maximum changes from m to m', rescale both ℓ and a by $e^{m-m'}$. Then add the new exponentials and value contributions under the new reference.

At the end,

$$o = \frac{a}{\ell}.$$

For attention, x_j is not a fixed input vector stored in advance. Each block of logits is generated on demand from a matrix product $Q_i K_j^\top / \sqrt{d}$ plus mask/bias terms. The online recurrence lets the kernel consume that block immediately.

The same idea works row by row and block by block, which is what makes a tiled exact softmax-attention forward pass possible.

10 分块合并（Blockwise Merge） 重复

对应原文第 14 页；点击查看相邻原文对照页

10 分块合并（Blockwise Merge） 循环 对于一个 查询（Query） 行，假设一些 键（Key） 块之后的运行状态是

其中 m 是迄今为止看到的最大分数，ℓ 是该最大值下稳定的 Softmax 归一化（Softmax） 分母，a∈ Rdv 是未归一化的 值（Value） 累加器（Accumulator）。

对于具有匹配值Vb∈Rb×dv的新得分块εRb，令mb = max(s), m ′ = max(m,m b),

然后更新

最后，

对于 查询（Query） 行的块，m 和 ℓ 变为按行 向量（Vectors），a 变为 矩阵（Matrix）。可以在指数之前将掩码应用于分数图块，掩码位置的概率为零。

递归是瓦片边界不会改变密集 Softmax 归一化（Softmax） 结果的代数原因。不同的 分块（Tiling） 会更改 浮点（Floating-point） 运算的顺序，但不会更改预期的实数算术函数。

FlashAttention 发布的算法以块形式表达等效的 running-max / running-normalizer / 输出更新。 [4, 5]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

10 The blockwise merge recurrence

For one query row, suppose the running state after some key blocks is

$$(m, \ell, a),$$

where m is the maximum score seen so far, ℓ is the stable softmax denominator under that maximum, and $a \in \mathbb{R}^{d_v}$ is the unnormalized value accumulator.

For a new score block $s \in \mathbb{R}^b$ with matching values $V_b \in \mathbb{R}^{b \times d_v}$, let

$$m_b = \max(s), \qquad m' = \max(m, m_b),$$

$$\alpha = e^{m - m'}, \qquad p = e^{s - m'}.$$

Then update

$$\ell' = \alpha \ell + \sum_r p_r,$$

$$a' = \alpha a + p^\top V_b,$$

$$m \leftarrow m', \quad \ell \leftarrow \ell', \quad a \leftarrow a'.$$

Finally,

$$o = a / \ell.$$

For a block of query rows, m and ℓ become row-wise vectors and a becomes a matrix. Masks can be applied to the score tile before the exponentials, with masked positions contributing zero probability.

> The recurrence is the algebraic reason tile boundaries do not change the dense softmax result. A different tiling changes the order of floating-point operations, but not the intended real-arithmetic function.

FlashAttention's published algorithms express equivalent running-max / running-normalizer / output updates in block form.[4, 5]

11 一个完整的数值例子

对应原文第 15 页；点击查看相邻原文对照页

11 一个完整的数值示例 考虑一个已缩放、未屏蔽的 注意力机制（Attention） 行

带二维 值（Values）

全局最大值为4，因此稳定的非标准化权重为

他们的总和是

完整的 Softmax 归一化（Softmax） 给出

现在处理两个块。对于[2, 1]:

对于第二个块[4, 3]，新的最大值是4，所以

然后

因此

匹配全行计算。

旧街区没有被重新访问。当出现更大的最大值时，它的贡献只是重新调整。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

11 A complete numerical example

Consider one already-scaled, unmasked attention row

$$s = [2, 1, 4, 3]$$

with two-dimensional values

$$v_1 = [1, 0],\ v_2 = [0, 1],\ v_3 = [2, 0],\ v_4 = [0, 2].$$

The global maximum is 4, so stable unnormalized weights are

$$[e^{-2}, e^{-3}, 1, e^{-1}] \approx [0.135335, 0.049787, 1, 0.367879].$$

Their sum is

$$\ell \approx 1.55300179,$$

and full softmax gives

$$O \approx [1.37497284,\ 0.50582424].$$

Now process two blocks. For $[2, 1]$:

$$m_1 = 2, \quad \ell_1 = 1 + e^{-1} = 1.36787944, \quad a_1 = [1, e^{-1}].$$

For the second block $[4, 3]$, the new maximum is 4, so

$$\alpha = e^{2-4} = e^{-2}.$$

Then

$$\ell_2 = \alpha\ell_1 + 1 + e^{-1} = 1.55300179,$$

$$a_2 = \alpha a_1 + [2, 2e^{-1}] \approx [2.13533528, 0.78554595].$$

Therefore

$$a_2/\ell_2 \approx [1.37497284, 0.50582424],$$

matching the full-row computation.

> The old block was not revisited. Its contribution was merely rescaled when a larger maximum appeared.

12 FlashAttention 前向传播（Forward Pass）

对应原文第 16 页；点击查看相邻原文对照页

12 FlashAttention 前向传播（Forward Pass） 一个有用的概念 前向传播（Forward Pass） 是：

1. 将 Q 划分为 查询（Query） 行瓷砖，将 K、V 划分为 键（Key）/值（Value） 瓷砖。

2. 对于每个 查询（Query） 图块，初始化逐行运行最大值、标准化器和输出累加器。

3.加载键（Key）/值（Value）图块并形成本地分数图块QiK⊤

4. 应用属于此图块的因果/局部掩模或附加偏差。

5. 计算图块最大值，更新运行最大值，并重新调整先前状态。

6. 相对于更新的最大值对当前图块求幂。

7. 更新分母和 值（Value） 加权输出 累加器（Accumulator）。

8. 继续，直到每个所需的 键（Key） 块都已贡献。

9. 按行标准化 累加器（Accumulator） 并写入输出。

原始的 FlashAttention 算法选择块大小，以便相关图块和状态适合片上 SRAM，从而减少 HBM 的行程。[4]

教育实现可以在 PyTorch 的几行中重现代数，但这样的代码不是高 性能（Performance） FlashAttention 内核（Kernel）。生产实现取决于 GPU 特定的 分块（Tiling）、线程/线程束（Warp） 调度、异步副本、张量（Tensor）-核心（Core） 指令以及其他低级细节。

基本的算法思想独立于一个 CUDA 内核（Kernel）：生成一个分数图块，通过 在线 Softmax（Online Softmax） 和 V 累加（Accumulation） 立即使用它，并且永远不会在 HBM 中实现完整的分数/概率 矩阵（Matrix）。

最小教育 PyTorch 下面的代码流 K/V 块并直接实现递归。它省略了 批处理（Batching）、掩码、随机失活（Dropout）、查询（Query） 分块（Tiling） 和每个 GPU 优化，因此它是正确性说明，而不是 FlashAttention 内核（Kernel）。

比例 = 1 / math.sqrt(q.shape[-1]) m = q.new_full((q.shape[0],), - float("inf")) l = q.new_zeros(q.shape[0]) a = q.new_zeros((q.shape[0], v.shape[-1]))

kj, vj = k[j:j+block], v[j:j+block] s = (q @ kj.T) * 缩放比例 m_new = torch.maximum(m, s.max(dim=-1).值（Values）) alpha = torch.exp(m - m_new) p = torch.exp(s - m_new[:, None]) a = a * alpha[:, None] + p @ vj l = l * 阿尔法 + p。 sum(dim=-1) m = m_new

后来的 FlashAttention 代际保留了这种语义结构，同时改变了在较新硬件上安排工作的方式。[5,6,7]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

12 FlashAttention forward pass

A useful conceptual forward pass is:

1. Partition Q into query-row tiles and K, V into key/value tiles.
2. For each query tile, initialize row-wise running maxima, normalizers, and output accumulators.
3. Load a key/value tile and form the local score tile $Q_i K_j^\top / \sqrt{d}$.
4. Apply causal/local masks or additive biases that belong to this tile.
5. Compute the tile maximum, update the running maximum, and rescale previous state.
6. Exponentiate the current tile relative to the updated maximum.
7. Update the denominator and the value-weighted output accumulator.
8. Continue until every required key tile has contributed.
9. Normalize the accumulator row-wise and write the output.

The original FlashAttention algorithm chooses block sizes so the relevant tiles and state fit in on-chip SRAM, reducing trips to HBM.[4]

An educational implementation can reproduce the algebra in a few lines of PyTorch, but such code is *not* a high-performance FlashAttention kernel. Production implementations depend on GPU-specific tiling, thread/warp scheduling, asynchronous copies, tensor-core instructions, and other low-level details.

> The essential algorithmic idea is independent of one CUDA kernel: generate a score tile, consume it immediately through online softmax and V accumulation, and never materialize the complete score/probability matrix in HBM.

Minimal educational PyTorch

The code below streams K/V blocks and implements the recurrence directly. It omits batching, masks, dropout, query tiling, and every GPU optimization, so it is a correctness illustration rather than a FlashAttention kernel.

```
def tiled_attention(q, k, v, block=128):
    scale = 1 / math.sqrt(q.shape[-1])
    m = q.new_full((q.shape[0],), -float("inf"))
    l = q.new_zeros(q.shape[0])
    a = q.new_zeros((q.shape[0], v.shape[-1]))
    for j in range(0, k.shape[0], block):
        kj, vj = k[j:j+block], v[j:j+block]
        s = (q @ kj.T) * scale
        m_new = torch.maximum(m, s.max(dim=-1).values)
        alpha = torch.exp(m - m_new)
        p = torch.exp(s - m_new[:, None])
        a = a * alpha[:, None] + p @ vj
        l = l * alpha + p.sum(dim=-1)
        m = m_new
    return a / l[:, None]
```

Later FlashAttention generations keep this semantic structure while changing how work is scheduled on newer hardware.[5, 6, 7]

13 为什么密集FlashAttention是准确的

对应原文第 17 页；点击查看相邻原文对照页

13 为什么密集FlashAttention是精确的 密集FlashAttention被称为精确是因为目标函数仍然存在

分块（Tiling） 不会删除 查询（Query）-键（Key） 对。 在线 Softmax（Online Softmax） 不会用其他函数替换指数或 归一化（Normalization）。块循环只是改变了积累足够统计数据的顺序。[4]

三个注意事项使这个词准确无误：

- 浮点（Floating Point） 是有限的精度（Precision）。重新排序添加和减少可能会产生与其他实现的微小数值差异。 PyTorch 明确警告 SDPA 后端可能会有所不同，因为 浮点（Floating-point） 操作的融合和顺序不同。[10, 13]

- 随机失活（Dropout） 在训练（Training） 期间是随机的。逐位比较两次运行除了数学注意力机制（Attention） 语义之外还需要匹配随机行为。

- 稀疏变体是不同的。原始FlashAttention 论文还提出了块稀疏FlashAttention，它省略了块，因此是相对于完整稠密注意力（Dense Attention） 的近似/稀疏变体。[4]

“精确”并不意味着“与每个参考 内核（Kernel） 按位相同”。这意味着该算法并没有故意改变密集的 Softmax 归一化（Softmax） 注意力机制（Attention） 以减少数学工作。

在评估数值测试时，这种区别很重要。合理的容差取决于类型、累加（Accumulation） 顺序、序列长度和后端，而不需要二进制同一性。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

13 Why dense FlashAttention is exact

Dense FlashAttention is called exact because the target function remains

$$\text{softmax}(QK^\top/\sqrt{d} + B)V.$$

Tiling does not delete query-key pairs. Online softmax does not replace the exponential or normalization with another function. The block recurrence simply changes the order in which sufficient statistics are accumulated.[4]

Three caveats keep the word **exact** precise:

- **Floating point is finite precision.** Reordering additions and reductions can produce small numerical differences from another implementation. PyTorch explicitly warns that SDPA backends can differ because floating-point operations are fused and ordered differently.[10, 13]
- **Dropout is stochastic during training.** Comparing two runs bit-for-bit requires matching random behavior in addition to mathematical attention semantics.
- **Sparse variants are different.** The original FlashAttention paper also presents block-sparse FlashAttention, which omits blocks and is therefore an approximate/sparse variant relative to full dense attention.[4]

> "Exact" does not mean "bitwise identical to every reference kernel." It means the algorithm is not intentionally changing dense softmax attention to reduce the mathematical work.

This distinction matters when evaluating numerical tests. A sensible tolerance depends on dtype, accumulation order, sequence length, and backend rather than requiring binary identity.

14 输入输出复杂度（IO Complexity）：定理实际上说了什么

对应原文第 18 页；点击查看相邻原文对照页

14 输入输出复杂度（IO Complexity）：定理的实际含义 最初的 FlashAttention 论文分析了具有 HBM 和大小为 M 的片上 SRAM 的两级存储器 模型（Model）。根据论文的假设，包括磁头尺寸和

标准具体化 注意力机制（Attention） 需要

HBM 访问，而 FlashAttention 需要

HBM 访问。该论文还证明了 模型（Model） 中一系列 SRAM 大小的最优结果。[4]

有几个细节很重要：

- 这些是特定内存模型（Model） 中的IO 复杂性结果，而不是每个GPU 的通用字节计数。

- 数量计算标量元素/字在建模内存级别之间的移动，直至渐近因子。

- 这不是算术复杂性。 稠密注意力（Dense Attention） 仍然执行 O(N 2d) 工作。

- 增加可用的片上存储器M 可实现更多的重用并减少建模的HBM 流量。

一个常见的错误总结是“FlashAttention 将 注意力机制（Attention） IO 从 O(N 2) 减少到 O(N)”。线性量是关于序列长度的大辅助内存占用（Memory Footprint），而不是上面一般的HBM-access表达式。

直观地说，更多可用的片上内存可以让更大的工作块保留在 驻留（Resident） 中，并在必须从 HBM 重新加载数据之前重复用于更多的 注意力机制（Attention） 工作。在该定理的体系中，增加的重用是 HBM 访问界限随着 M 的增长而减小的原因。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

14 IO complexity: what the theorem actually says

The original FlashAttention paper analyzes a two-level memory model with HBM and on-chip SRAM of size M. Under the paper's assumptions, including head dimension d and

$$d \le M \le Nd,$$

standard materializing attention requires

$$\Theta(Nd + N^2)$$

HBM accesses, whereas FlashAttention requires

$$\Theta\left(\frac{N^2 d^2}{M}\right)$$

HBM accesses. The paper also proves an optimality result over a range of SRAM sizes in this model.[4]

Several details matter:

- These are **IO-complexity results in a particular memory model**, not a universal byte count for every GPU.
- The quantity counts movement of scalar elements/words between the modeled memory levels, up to asymptotic factors.
- It is not the arithmetic complexity. Dense attention still performs $O(N^2 d)$ work.
- Increasing usable on-chip memory M enables more reuse and reduces modeled HBM traffic.

> A common incorrect summary is "FlashAttention reduces attention IO from $O(N^2)$ to $O(N)$." The linear quantity is the large *auxiliary memory footprint* with respect to sequence length, not the general HBM-access expression above.

Intuitively, more usable on-chip memory lets larger working tiles stay resident and be reused for more attention work before data must be reloaded from HBM. In the theorem's regime, that increased reuse is why the HBM-access bound decreases as M grows.

15 内存复杂度：线性辅助状态，非线性计算

对应原文第 19 页；点击查看相邻原文对照页

15 内存复杂性：线性辅助状态，而不是线性计算 为什么FlashAttention论文经常说内存变成线性而不是二次?

输入和输出已经包含O(Nd)个元素。具体化的 稠密注意力（Dense Attention） 实现另外创建 O(N 2) 分数/概率状态。 FlashAttention 避免存储那些完整的 矩阵（Matrices） 并仅保留平铺工作状态以及按行统计信息。因此，与 注意力机制（Attention） 操作相关的额外内存随序列长度线性扩展，而不是二次扩展。[4, 5]

对于 训练（Training），差异尤其重要，因为简单的 反向传播（Backward Pass） 可能会保存完整的概率矩阵 P。 FlashAttention 而是保存紧凑信息，例如输出和逐行 归一化（Normalization） 统计信息，然后向后重新计算分数/概率图块。[4, 5]

线性内存并不意味着线性运行时间。密集的 查询（Query）-键（Key） 交互的数量仍然是 N 的二次方。

还要避免声称 模型（Model） 内存总量为 O(N)。其他 Transformer 模型（Transformer） 组件消耗 激活（Activation） 内存，而自回归 推理服务（Serving） 具有随上下文长度增长的 KV 缓存内存。
FlashAttention 特别改变了 注意力机制（Attention） 计算管理其中间体的方式。

这是 FlashAttention 和 激活（Activation） 检查点可以共存的原因之一：两者都使用 重计算（Recomputation） 来减少存储状态，但在 训练（Training） 图的不同范围内。

15 Memory complexity: linear auxiliary state, not linear compute

Why do FlashAttention papers often say memory becomes linear instead of quadratic?

The inputs and output already contain $O(Nd)$ elements. A materializing dense attention implementation additionally creates $O(N^2)$ score/probability state. FlashAttention avoids storing those full matrices and retains only tiled working state plus row-wise statistics. As a result, the extra memory associated with the attention operation scales linearly with sequence length rather than quadratically.[4, 5]

For training, the difference is especially important because a straightforward backward pass might otherwise save a full probability matrix P. FlashAttention instead saves compact information such as output and row-wise normalization statistics, then recomputes score/probability tiles in backward.[4, 5]

> **Linear memory does not imply linear runtime.** The number of dense query-key interactions is still quadratic in N.

Also avoid claiming that total model memory is $O(N)$. Other Transformer components consume activation memory, and autoregressive serving has KV-cache memory that grows with context length. FlashAttention is specifically changing how the attention computation manages its intermediates.

This is one reason FlashAttention and activation checkpointing can coexist: both trade recomputation for reduced stored state, but at different scopes of the training graph.

16 因果掩蔽和瓦片跳跃

对应原文第 20 页；点击查看相邻原文对照页

16 因果屏蔽和跳格 在 因果自注意力（Causal Self-attention） 中，查询（Query） 位置 i 可能不会关注未来的 键（Key） 位置 j >i。物化的 掩码（Mask） 将是另一个 N×N 对象，但高效的平铺 内核（Kernel） 可以推理每个平铺的几何形状。

对于方形 自注意力（Self-attention）：

- 严格高于因果边界的块被完全屏蔽并且不需要贡献，

- 严格低于边界的块是完全有效的，

- 只有与对角线相交的块才需要元素级因果屏蔽。

这可以避免在因果 内核（Kernel） 中计算许多无效的图块。 FlashAttention-2 显式地利用了因果结构，而当前的实现还为不相等的 查询（Query）/键（Key） 长度定义了精确的对齐规则。 [5,

掩码（Mask） 语义是 API 细节，不得猜测。例如，当前的 PyTorch SDPA 将 True 的布尔值 attn_掩码（Mask） 值（Value） 视为参与 注意力机制（Attention） 的位置，而其他 PyTorch 掩码（Mask） API 使用不同的约定。 [10]

本地/滑动窗口 注意力机制（Attention） 可以类似地跳过允许窗口之外的图块。但是，一旦 模型（Model） 有意限制参加的配对，该模型的 注意力机制（Attention） 模式就会稀疏/局部。 FlashAttention 可以是用于执行该模式的 内核（Kernel），但它不再是与不受限制的 稠密注意力（Dense Attention） 相同的数学问题。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

16 Causal masking and tile skipping

In causal self-attention, query position i may not attend to future key positions $j > i$. A materialized mask would be another $N \times N$ object, but an efficient tiled kernel can reason about the geometry of each tile.

For square self-attention:

- blocks strictly above the causal boundary are fully masked and need not contribute,
- blocks strictly below the boundary are fully valid,
- only blocks intersecting the diagonal require element-level causal masking.

This avoids computing many invalid tiles in a causal kernel. FlashAttention-2 explicitly exploits causal structure, while current implementations also define precise alignment rules for unequal query/key lengths.[5, 8]

> Mask semantics are an API detail that must not be guessed. For example, current PyTorch SDPA treats a Boolean `attn_mask` value of `True` as a position that *participates* in attention, while other PyTorch mask APIs use different conventions.[10]

Local/sliding-window attention can similarly skip tiles outside the permitted window. But once the model intentionally restricts which pairs are attended, the *model's attention pattern* is sparse/local. FlashAttention can be the kernel used to execute that pattern, but it is no longer the same mathematical problem as unrestricted dense attention.

17 反向传播（Backward Pass）：重新计算而不是存储

对应原文第 21 页；点击查看相邻原文对照页

17 反向传播（Backward Pass）：重新计算而不是存储 简单的 训练（Training） 实现可以保存 P = Softmax 归一化（Softmax）(S) 以进行向后操作。 FlashAttention 避免将 N 2 张量（Tensor） 保留在 HBM 中。相反，它存储紧凑的逐行 归一化（Normalization） 信息，并在需要梯度时重新计算分数/概率图块。 [4, 5]

对于

一个有用的行式恒等式是

（dOir）Oir。

重新计算 P 的瓦片后，局部导数可以表示为

那么 dQ +=dSK/

掩码意味着掩码条目的概率为零/梯度（Gradient） 贡献。

FA2 存储逐行 log-sum-exp 数量，允许根据重新计算的分数稳定地重建概率图块。 [5]

向后的 重计算（Recomputation） 是故意的。它花费额外的 矩阵（Matrix） 乘法工作来避免读写大概率的 张量（Tensor），这对 GPU 来说是一个有利的交易。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

17 Backward pass: recompute instead of store

A naïve training implementation can save $P = \mathrm{softmax}(S)$ for backward. FlashAttention avoids keeping that N^2 tensor in HBM. Instead, it stores compact row-wise normalization information and recomputes score/probability tiles when gradients are needed.[4, 5]

For

$$S = QK^\top/\sqrt{d}, \qquad P = \mathrm{softmax}(S), \qquad O = PV,$$

a useful row-wise identity is

$$D_i = \sum_r (dO_{ir})O_{ir}.$$

After recomputing a tile of P, the local derivatives can be expressed as

$$dV \mathrel{+}= P^\top dO,$$

$$dP = dO\,V^\top,$$

$$dS = P \odot (dP - D[:,\mathrm{None}]),$$

then

$$dQ \mathrel{+}= dS\,K/\sqrt{d}, \qquad dK \mathrel{+}= dS^\top Q/\sqrt{d}.$$

Masks imply zero probability/gradient contribution for masked entries.

FA2 stores a row-wise log-sum-exp quantity that allows the probability tile to be reconstructed stably from recomputed scores.[5]

> Backward recomputation is intentional. It spends extra matrix-multiply work to avoid reading and writing a giant probability tensor, which can be a favorable trade on GPUs.

18 为什么更多的浮点运算次数（FLOPs）仍然可以更快

对应原文第 22 页；点击查看相邻原文对照页

18 为什么更多的浮点运算次数（FLOPs） 仍然可以更快 人们很容易认为更少的算术运算总是意味着更低的延迟（Latency）。加速器 性能（Performance） 经常打破这种直觉。

矩阵乘法（Matrix Multiplication） 与 张量核心（Tensor Cores） 的映射非常好。 HBM 流量、同步（Synchronization）、共享内存流量、指数、缩减和 内核启动（Kernel Launch） 边界可能相对昂贵。

因此，FlashAttention 会进行一些交易，例如向后重新计算图块或进行重新缩放算术（如果这会阻止更大的内存传输）。[4, 5]

FA2 明确了这一对比：其目标之一是减少非 矩阵乘法（Matmul） 浮点运算次数（FLOPs），因为这些操作不享受与 张量（Tensor）-核心（Core） GEMM 相同的 吞吐量（Throughput）。该论文还改进了 工作划分（Work Partitioning），因此占用了更多的 GPU。[5]

FA3 在 Hopper 上更进一步，使用异步硬件功能重叠 矩阵乘法（Matrix Multiplication）、Softmax 归一化（Softmax） 和 数据移动（Data Movement）。 FA4 响应 Blackwell，其中 张量核心吞吐量（Tensor-core Throughput） 比其他一些资源增长得更快，使得指数和共享 内存流量（Memory Traffic） 相对更加重要。[6, 7]

更好的 性能（Performance） 问题不仅仅是“有多少个 浮点运算次数（FLOPs）？”但是“哪些操作、哪些单元、使用什么 数据移动（Data Movement）、重用、并行性和 同步（Synchronization）？”这是使 FlashAttention 变得比 注意力机制（Attention） 本身更重要的系统课程：硬件效率通常来自于与内存/执行层次结构的共同设计数学调度。

18 Why more FLOPs can still be faster

It is tempting to assume that fewer arithmetic operations always imply lower latency. Accelerator performance breaks that intuition regularly.

Matrix multiplication maps exceptionally well to tensor cores. HBM traffic, synchronization, shared-memory traffic, exponentials, reductions, and kernel launch boundaries can be comparatively expensive. FlashAttention therefore makes trades such as recomputing tiles in backward or doing rescaling arithmetic if that prevents much larger memory transfers.[4, 5]

FA2 makes this contrast explicit: one of its goals is to reduce **non-matmul FLOPs**, because those operations do not enjoy the same throughput as tensor-core GEMMs. The paper also improves work partitioning so more of the GPU is occupied.[5]

FA3 goes further on Hopper by overlapping matrix multiplication, softmax, and data movement using asynchronous hardware features. FA4 responds to Blackwell, where tensor-core throughput increased faster than some other resources, making exponentials and shared-memory traffic relatively more important.[6, 7]

> A better performance question is not merely "How many FLOPs?" but "Which operations, on which units, with what data movement, reuse, parallelism, and synchronization?"

This is the systems lesson that makes FlashAttention important beyond attention itself: hardware efficiency often comes from co-designing mathematical scheduling with the memory/execution hierarchy.

19 MHA、MQA 和 GQA 兼容性

对应原文第 23 页；点击查看相邻原文对照页

19 MHA、MQA 和 GQA 兼容性 FlashAttention 并不要求每个架构都具有相同数量的 查询（Query） 和 键值头（Key/Value Heads）。

让

对于普通MHA，Hq =Hkv。在 MQA 中，Hkv = 1。在 GQA 中，1<H kv <H q。当前Dao-AILab 内核（Kernels）通过传递比查询头（Query Heads）更少的键值头（KV Heads）来支持MQA/GQA，并要求查询头（Query Heads）的数量可以被键值头（KV Heads）的数量整除。[8]

内核（Kernel） 仍然评估每个 查询头（Query Head） 与其分配的 KV 头之间的 注意力机制（Attention）。头共享改变了架构和KV-内存占用（Memory Footprint）。 FlashAttention 更改生成的 注意力机制（Attention） 操作的执行方式。

正交概念：

- MQA/GQA：头（Heads） 如何共享 K/V 投影，

- RoPE：位置如何变换Q/K，

- FlashAttention：如何高效地调度和计算注意力机制（Attention）。

这些可以一起使用。

当前的 PyTorch SDPA 还暴露senable_gqa。其文档将 GQA 支持标记为实验性的，并施加后端和 张量（Tensor） 形状约束，因此生产代码应遵循确切版本的文档，而不是假设通用融合 内核（Kernel） 支持。[10]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

19 MHA, MQA, and GQA compatibility

FlashAttention does not require every architecture to have the same number of query and key/value heads.

Let

$$Q \in \mathbb{R}^{B\times N_q\times H_q\times d}, \qquad K,V \in \mathbb{R}^{B\times N_k\times H_{kv}\times d}.$$

For ordinary MHA, $H_q = H_{kv}$. In MQA, $H_{kv} = 1$. In GQA, $1 < H_{kv} < H_q$. Current Dao-AILab kernels support MQA/GQA by passing fewer KV heads than query heads, with the requirement that the number of query heads be divisible by the number of KV heads.[8]

The kernel still evaluates attention between each query head and its assigned KV head. Head sharing changes the architecture and KV-memory footprint. FlashAttention changes how the resulting attention operation is executed.

Orthogonal concepts:

- MQA/GQA: how heads share K/V projections,
- RoPE: how position transforms Q/K,
- FlashAttention: how attention is scheduled and computed efficiently.

These can be used together.

Current PyTorch SDPA also exposes `enable_gqa`. Its documentation labels GQA support experimental and imposes backend- and tensor-shape constraints, so production code should follow the exact version's documentation rather than assuming universal fused-kernel support.[10]

20 个可变长度、局部注意力（Local Attention） 和 随机失活（Dropout）

对应原文第 24 页；点击查看相邻原文对照页

20 可变长度、局部注意力（Local Attention） 和 随机失活（Dropout） 生产 注意力机制（Attention） 很少仅由等长密集序列组成，不含 随机失活（Dropout）。当前的 FlashAttention 实现支持更广泛的功能集，但这些是实现功能，而不是数学思想本身的属性。

当前的 Dao-AILab 存储库记录了 内核（Kernels）/interfaces 的功能，包括：

- 可变长度序列，

- 因果注意力（Causal Attention），

- 本地/滑动窗口 注意力机制（Attention），

- 随机失活（Dropout） 位于面向 训练（Training） 的接口中，

-MQA/GQA，

- 相关界面中的 ALiBi 式评分偏差，

- 使用 键值缓存（KV Cache） 的专用增量解码路径，包括可选的 RoPE 处理。[8]

功能支持因 CUDA/ROCm 后端而异，并随着时间的推移而发展。例如，存储库记录了具有不同实现细节的独立 NVIDIA 和 AMD 后端，并支持 矩阵（Matrices）。 [8]

随机失活（Dropout） 值得实际警告。当前 PyTorchscaled_dot_product_注意力机制（Attention） 始终根据其 dropout_p 参数应用 随机失活（Dropout），因此当应禁用 随机失活（Dropout） 时，调用者必须在评估期间传递 0.0。 [10]

请勿从名称“FlashAttention”推断功能支持。检查实际运行的确切库、内核（Kernel） 代际（Generation）、设备、dtype、头维度（Head Dimension）、掩码（Mask）/bias 和 训练（Training）/推理（Inference） 路径。

20 Variable lengths, local attention, and dropout

Production attention rarely consists only of equal-length dense sequences with no dropout. Current FlashAttention implementations support a broader feature set, but these are **implementation capabilities**, not properties of the mathematical idea itself.

The current Dao-AILab repository documents kernels/interfaces for features including:

- variable-length sequences,
- causal attention,
- local/sliding-window attention,
- dropout in training-oriented interfaces,
- MQA/GQA,
- ALiBi-style score bias in relevant interfaces,
- specialized incremental-decoding paths with KV cache, including optional RoPE handling.[8]

Feature support differs by CUDA/ROCm backend and evolves over time. For example, the repository documents separate NVIDIA and AMD backends with different implementation details and support matrices.[8]

Dropout deserves a practical warning. Current PyTorch `scaled_dot_product_attention` always applies dropout according to its `dropout_p` argument, so callers must pass 0.0 during evaluation when dropout should be disabled.[10]

> Do not infer feature support from the name "FlashAttention." Check the exact library, kernel generation, device, dtype, head dimension, mask/bias, and training/inference path that will actually run.

21 FlashAttention-2：发生了什么变化

对应原文第 25 页；点击查看相邻原文对照页

21 FlashAttention-2：FlashAttention-2 的改变并没有改变根本目标。它仍然计算 精确注意力（Exact Attention），同时避免二次物化中间体。改进主要是为了更好地使用 GPU。[5]

FA2 论文确定了三个主要变化：

1.减少非矩阵乘法（Matmul） 浮点运算次数（FLOPs）的数量，

2. 即使对于单个头，也可以跨序列图块并行化 注意力机制（Attention），以改进 占用率（Occupancy），

3. 线程块（Thread Block） 内的 线程束（Warps） 之间的重新分区工作，以减少共享内存通信。[5]

在本文的 A100 实验中，这些变化比 FlashAttention-1 产生了大约 2 倍的加速，并达到理论最大值 浮点运算次数（FLOPs）/s 的 50-73%，而在报告的设置下，端到端 GPT 式 训练（Training） 达到每 A100 225 TFLOPs/s。 [5]

这些数字并不是普遍的速度保证。它们取决于 GPU、形状、数据类型、因果模式、软件版本和 基准测试（Benchmark） 方法。

FA1 的突破在于 IO 感知平铺 精确注意力（Exact Attention）。 FA2 的突破在于更好的并行性和 工作划分（Work Partitioning） 围绕相同的高级算法思想。

这种模式将会重复：后代保留语义，但使 内核（Kernel） 流水线（Pipeline） 适应较新 GPU 架构的资源和瓶颈。

21 FlashAttention-2: what changed

FlashAttention-2 did not change the fundamental goal. It still computes exact attention while avoiding quadratic materialized intermediates. The improvement was mostly about using the GPU better.[5]

The FA2 paper identifies three main changes:

1. reduce the number of non-matmul FLOPs,
2. parallelize attention across sequence tiles even for a single head to improve occupancy,
3. repartition work among warps within a thread block to reduce shared-memory communication.[5]

In the paper's A100 experiments, these changes produced roughly a 2× speedup over FlashAttention-1 and reached 50–73% of theoretical maximum FLOPs/s, while end-to-end GPT-style training reached up to 225 TFLOPs/s per A100 under the reported setup.[5]

Those numbers are not universal speed guarantees. They depend on GPU, shapes, dtype, causal mode, software versions, and benchmark methodology.

> FA1's breakthrough was IO-aware tiled exact attention. FA2's breakthrough was better parallelism and work partitioning around the same high-level algorithmic idea.

This pattern will repeat: later generations keep the semantics but adapt the kernel pipeline to the resources and bottlenecks of newer GPU architectures.

22 FA2 跨序列图块的并行性

对应原文第 26 页；点击查看相邻原文对照页

22 FA2 跨序列图块的并行性 GPU 需要足够的独立工作来保持许多流式多处理器繁忙。在 FA1 中，并行性的自然单位大致是批×头。如果 批大小（Batch Size） 和人数较少，即使序列本身很长，也可能导致硬件利用率不足。

FA2 沿序列维度添加并行性。不同的线程块可以在同一 注意力头（Attention Head） 的不同 查询（Query） 行图块上工作。由于 查询（Query） 行在其自己的 键（Key）/值（Value） 缩减完成之前是独立的，因此这可以在不更改 注意力机制（Attention） 语义的情况下公开更多并行工作。 [5]

一个简化的心理图景是：

Q rows block 0 block 1 block 2 block 3 block 4 block 5 CTA CTA CTA 重要的一点不是确切的块分配，这是内核（Kernel） 特定的。正是长序列长度本身成为并行性的来源。

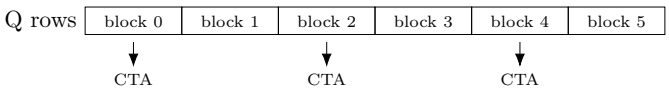
并行块越多并不一定越好。切片大小、占用率（Occupancy）、寄存器（Registers）、共享内存（Shared Memory） 和数据重用相互作用。 内核（Kernel）设计是对硬件资源的优化问题，而不仅仅是最大线程数的请求。

22 FA2 parallelism across sequence tiles

A GPU needs enough independent work to keep many streaming multiprocessors busy. In FA1, a natural unit of parallelism was roughly batch × head. If batch size and head count were small, that could leave hardware under-occupied even when the sequence itself was long.

FA2 adds parallelism along the sequence dimension. Different thread blocks can work on different query-row tiles for the same attention head. Because query rows are independent until their own key/value reductions are complete, this exposes more parallel work without changing attention semantics.[5]

A simplified mental picture is:



The important point is not the exact block assignment, which is kernel-specific. It is that long sequence length itself becomes a source of parallelism.

> More parallel blocks are not automatically better. Tile size, occupancy, registers, shared memory, and data reuse interact. Kernel design is an optimization problem over hardware resources, not just a request for maximum thread count.

23 FA2 工作划分（Work Partitioning） 和非 矩阵乘法（Matmul） 浮点运算次数（FLOPs）

对应原文第 27 页；点击查看相邻原文对照页

23 FA2 工作划分（Work Partitioning） 和非 矩阵乘法（Matmul） 浮点运算次数（FLOPs） FA2 还改变了每个 线程块（Thread Block） 内部的工作。 FA1 的 线程束（Warp） 分区可能需要将部分结果写入 共享内存（Shared Memory） 并进行组合，从而产生通信开销。 FA2 重新分区工作，因此 线程束（Warps） 以减少共享内存读/写的方式进行协作。[5]

该论文还减少了相对于 张量（Tensor）-核心（Core） 矩阵乘法（Matrix Multiplication） 昂贵的操作。这包括围绕 Softmax 归一化（Softmax） 重新缩放和向后计算进行代数重组，从而减少执行的非 矩阵乘法（Matmul） 浮点运算次数（FLOPs）。 [5]

这很重要，因为“一个 浮点运算（FLOP）”并不是所有操作类型中有用的 性能（Performance） 货币。 张量核心（Tensor Cores） 可以以巨大的 吞吐量（Throughput） 执行 矩阵（Matrix） 乘法累加，而指数、标量缩减、转换和共享内存 同步（Synchronization） 有不同的限制。

FA2提醒的是，IO改进后，下一个瓶颈可能是占用率（Occupancy）、线程束（Warp）通信，或者非矩阵乘法（Matmul）工作。优化是移动瓶颈，而不是使硬件限制消失。

FA2 还细化了向后保存的内容，使用紧凑的行对数和表达式信息并重新计算图块而不是具体化概率。这保留了线性内存特性，同时改善了 内核（Kernel） 的工作分配。[5]

23 FA2 work partitioning and non-matmul FLOPs

FA2 also changes work inside each thread block. FA1's warp partitioning could require partial results to be written to shared memory and combined, creating communication overhead. FA2 repartitions work so warps cooperate in a way that reduces shared-memory reads/writes.[5]

The paper also reduces operations that are expensive relative to tensor-core matrix multiplication. This includes algebraic restructuring around softmax rescaling and backward computations so fewer non-matmul FLOPs are performed.[5]

This matters because "one FLOP" is not a useful performance currency across all operation types. Tensor cores can execute matrix multiply-accumulate at enormous throughput, while exponentials, scalar reductions, conversions, and shared-memory synchronization have different limits.

> FA2 is a reminder that after IO is improved, the next bottleneck may be occupancy, warp communication, or non-matmul work. Optimization moves the bottleneck rather than making hardware constraints disappear.

FA2 also refines what is saved for backward, using compact row-wise log-sum-exp information and recomputing tiles instead of materializing probabilities. This preserves the linear-memory character while improving the kernel's work distribution.[5]

24 FlashAttention-3：料斗 代际（Generation）

对应原文第 28 页；点击查看相邻原文对照页

24 FlashAttention-3：Hopper 代际（Generation） Hopper GPU 引入的硬件功能改变了调度 注意力机制（Attention） 的最佳方式。 FlashAttention-3 是专门为利用这些功能而设计的。[6]

FA3 论文指出，FA2 在测量中仅在 H100 上实现了约 35% 的利用率，留下了巨大的空间。它提出了三个广泛的想法：

1. 使用异步和 线程束（Warp） 专业化的重叠计算和 数据移动（Data Movement），

2. 交错 矩阵乘法（Matrix Multiplication） 和 Softmax 归一化（Softmax） 工作，使不同的执行单元保持忙碌，

3. 利用旨在控制数值误差的 量化（Quantization） 技术来开发 FP8 硬件。 [6]

同样，这些是特定于硬件和 基准测试（Benchmark） 的经验结果，而不是对任意 模型（Models） 的承诺。

FA3 并不是“更近似的 FlashAttention”。 FP16/BF16 路径保留精确的 注意力机制（Attention） 算法目标。 FP8 路径有意引入较低的 精度（Precision） 算术，因此需要自己的数值 准确性（Accuracy） 讨论。

在 H100 上，NeurIPS 2024 最终出版物报告其 基准测试（Benchmark） 套件比 FA2 加速 1.5-2.0 倍，BF16 吞吐量（Throughput） 高达 840 TFLOPs/s（85% 利用率），FP8 吞吐量（Throughput） 达到 1.3 PFLOPs/s。 [6]

24 FlashAttention-3: the Hopper generation

Hopper GPUs introduced hardware features that changed the best way to schedule attention. FlashAttention-3 was designed specifically to exploit those capabilities.[6]

The FA3 paper points out that FA2 achieved only about 35% utilization on H100 in its measurements, leaving substantial headroom. It develops three broad ideas:

1. overlap computation and data movement using asynchrony and warp specialization,
2. interleave matrix multiplication and softmax work so different execution units remain busy,
3. exploit FP8 hardware with quantization techniques designed to control numerical error.[6]

On H100, the final NeurIPS 2024 publication reports 1.5-2.0x speedup over FA2 in its benchmark suite, BF16 throughput up to 840 TFLOPs/s (85% utilization), and FP8 throughput reaching 1.3 PFLOPs/s.[6]

Again, these are hardware- and benchmark-specific empirical results, not promises for arbitrary models.

> FA3 is not "a more approximate FlashAttention." The FP16/BF16 path preserves the exact-attention algorithmic goal. The FP8 path intentionally introduces lower-precision arithmetic and therefore needs its own numerical-accuracy discussion.

25 FA3 异步：重叠 数据移动（Data Movement）、通用矩阵乘法（GEMM） 和 Softmax 归一化（Softmax）

对应原文第 29 页；点击查看相邻原文对照页

25 FA3 异步：重叠 数据移动（Data Movement）、通用矩阵乘法（GEMM）、

在 Hopper 上，张量内存加速器（Tensor Memory Accelerator） (TMA) 和异步 线程束（Warp） 组 矩阵（Matrix） 乘法运算使得构建更深层次的软件 流水线（Pipeline） 成为可能。 FA3 使用 线程束（Warp） 专门化，因此不同的 线程束（Warps） 负责不同的阶段，允许 数据移动（Data Movement） 和计算重叠。[6]

简化的 流水线（Pipeline） 是：

加载下一个图块 // QK⊤ 通用矩阵乘法（GEMM） // Softmax 归一化（Softmax）/更新 // PV 通用矩阵乘法（GEMM）。

符号 // 表示这些阶段在依赖性允许的情况下有意重叠，而不是说它们在数学上是独立的。

FA3 还使用乒乓式时间表来交错块 矩阵乘法（Matrix Multiplication） 和 Softmax 归一化（Softmax）。重点是避免在执行标量/特殊功能工作时使 张量核心（Tensor Cores） 空闲，反之亦然。[6]

在线-Softmax 归一化（Softmax） 重现在概念上仍然存在。变化的是用于执行乐谱、归一化（Normalization） 和 值（Value）-累加（Accumulation） 阶段的硬件 流水线（Pipeline）。

这是该系列中的一个重要进展：FA1 询问“我们如何避免 HBM 流量？”，FA2 询问“我们如何更好地划分工作？”，FA3 询问“我们如何在 Hopper 上重叠不同的执行资源？”

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

25 FA3 asynchrony: overlap data movement, GEMM, and softmax

On Hopper, Tensor Memory Accelerator (TMA) and asynchronous warp-group matrix-multiply operations make it possible to build a deeper software pipeline. FA3 uses warp specialization so different warps take responsibility for different stages, allowing data movement and computation to overlap.[6]

A simplified pipeline is:

$$\text{load next tiles} \quad \| \quad QK^\top \text{ GEMM} \quad \| \quad \text{softmax/update} \quad \| \quad PV \text{ GEMM}.$$

The symbol $\|$ means the stages are intentionally overlapped where dependencies permit, not that they are mathematically independent.

FA3 also uses a ping-pong style schedule to interleave block matrix multiplication and softmax. The point is to avoid leaving tensor cores idle while scalar/special-function work is performed, and vice versa.[6]

> The online-softmax recurrence is still present conceptually. What changes is the hardware pipeline used to execute the score, normalization, and value-accumulation stages.

This is an important progression in the family: FA1 asks "How do we avoid HBM traffic?", FA2 asks "How do we partition the work better?", and FA3 asks "How do we overlap distinct execution resources on Hopper?"

26 FA3 FP8：性能（Performance） 不假装 精度（Precision） 是免费的

对应原文第 30 页；点击查看相邻原文对照页

26 FA3 FP8：性能（Performance） 无需假装 精度（Precision） 是免费的 Hopper 为 FP8 提供了非常高的张量核心吞吐量（Tensor-core Throughput）。然而，简单地将所有 注意力机制（Attention） 操作数转换为 FP8 可能会产生不可接受的数值错误，因为 注意力机制（Attention） 在因图块而异的动态范围上结合了点积、指数、归一化（Normalization） 和 值（Value） 累加（Accumulation）。

不要混淆“精确”的两个含义。 FlashAttention 的 分块（Tiling）/online-Softmax 归一化（Softmax） 算法对于 稠密注意力（Dense Attention） 公式来说可以是精确的，而以较低的 精度（Precision） 数字格式执行该公式仍然会引入 量化（Quantization）/舍入（Rounding） 错误。

模型（Model） 可接受的 精度（Precision） 取决于完整的 训练（Training）/推理（Inference） 设置。因此，FP8 支持是分层到 FlashAttention 执行策略上的硬件和数字功能，而不是证明 FP8 可以与 BF16/FP16 普遍互换。

官方存储库继续将 FA3 的 Hopper 特定实现与其他后端分开，这是不将 FA3 功能声明为“FlashAttention”通用属性的另一个原因。[8] FA3 引入了使用块 量化（Quantization） 的 FP8 路径和不连贯的处理技术，以在利用 FP8 张量核心吞吐量（Tensor-core Throughput） 的同时减少 量化误差（Quantization Error）。在 NeurIPS 2024 最终出版物中，FP8 FA3 在 H100 上达到 1.3 PFLOPs/s，并且报告的数值误差比用于比较的基线（Baseline） FP8 注意力机制（Attention） 方法低 2.6 倍。 [6]

26 FA3 FP8: performance without pretending precision is free

Hopper provides very high tensor-core throughput for FP8. Simply converting all attention operands to FP8, however, can produce unacceptable numerical error because attention combines dot products, exponentials, normalization, and value accumulation over dynamic ranges that vary by tile.

FA3 introduces an FP8 path using block quantization and an incoherent processing technique to reduce quantization error while exploiting FP8 tensor-core throughput. In the final NeurIPS 2024 publication, FP8 FA3 reaches 1.3 PFLOPs/s on H100 and reports 2.6x lower numerical error than the baseline FP8 attention method used for comparison.[6]

> Do not conflate two meanings of “exact.” FlashAttention’s tiling/online-softmax algorithm can be exact with respect to the dense attention formula, while performing that formula in a lower-precision numeric format still introduces quantization/rounding error.

A model’s acceptable precision depends on the full training/inference setup. FP8 support is therefore a hardware-and-numerics feature layered onto the FlashAttention execution strategy, not proof that FP8 is universally interchangeable with BF16/FP16.

The official repository continues to separate FA3’s Hopper-specific implementation from other backends, which is another reason not to state FA3 capabilities as generic properties of “FlashAttention.”[8]

27 FlashAttention-4：布莱克韦尔 代际（Generation）

对应原文第 31 页；点击查看相邻原文对照页

27 FlashAttention-4：Blackwell 代际（Generation） FlashAttention-4 延续了相同的硬件感知重新设计模式，这次是针对 NVIDIA Blackwell。

该论文认为，Blackwell 展示了对称硬件 缩放（Scaling）：张量核心吞吐量（Tensor-core Throughput） 相对于 Hopper 大约增加了一倍，而其他一些资源，包括共享 内存带宽（Memory Bandwidth） 和指数 吞吐量（Throughput），并没有以相同的速度扩展。 [7]

当一个子系统变得更快时，曾经次要的操作可能会成为瓶颈。

因此，FA4 重新设计了前向和后向管道，而不仅仅是重用 Hopper 调度表。

论文的主要技术包括：

-完全异步矩阵乘加（MMA）管道和更大的图块，

- 软件模拟指数加条件在线-Softmax 归一化（Softmax） 前向重新缩放，

- 张量（Tensor） 内存和 2-CTA 矩阵乘加（MMA） 技术可减少向后共享的 内存流量（Memory Traffic） 和原子添加。[7]

在带有 BF16 的 B200 上，该论文报告说，与 cuDNN 9.13 相比，速度提高了 1.3 倍，与 Triton 相比，速度提高了 2.7 倍，在评估的配置下达到 1613 TFLOPs/s（利用率为 71%）。 [7]

FA4 不会更改 稠密注意力（Dense Attention） 的大 O 算法。它攻击由 GPU 代际（GPU Generation） 创建的新瓶颈，其计算/内存/功能单元平衡与 Hopper 不同。

27 FlashAttention-4: the Blackwell generation

FlashAttention-4 continues the same pattern of hardware-aware redesign, this time for NVIDIA Blackwell. The paper argues that Blackwell exhibits **asymmetric hardware scaling**: tensor-core throughput roughly doubled relative to Hopper while some other resources, including shared-memory bandwidth and exponential throughput, did not scale at the same rate.[7]

When one subsystem becomes much faster, operations that used to be secondary can become the bottleneck. FA4 therefore redesigns both forward and backward pipelines rather than merely reusing the Hopper schedule.

The paper's major techniques include:

- fully asynchronous MMA pipelines and larger tiles,
- software-emulated exponential plus conditional online-softmax rescaling in forward,
- Tensor Memory and 2-CTA MMA techniques to reduce shared-memory traffic and atomic additions in backward.[7]

On B200 with BF16, the paper reports up to 1.3× speedup over cuDNN 9.13 and 2.7× over its Triton comparison, reaching up to 1613 TFLOPs/s (71% utilization) under the evaluated configurations.[7]

> FA4 does not change the big-O arithmetic of dense attention. It attacks the new bottlenecks created by a GPU generation whose compute/memory/function-unit balance differs from Hopper.

28 FA4 和非对称硬件 缩放（Scaling）

对应原文第 32 页；点击查看相邻原文对照页

28 FA4 和非对称硬件缩放（Scaling） FlashAttention 谱系是一个案例研究，说明为什么内核（Kernels）不能一次优化并假设永远最优。

在 Blackwell 上，矩阵乘法（Matrix Multiplication） 变得足够快，以至于 前向传播（Forward Pass） 可能会受到 Softmax 归一化（Softmax）/相对于 张量（Tensor）-核心（Core） 通用矩阵乘法（GEMM） 的指数功的约束。 FA4 使用软件模拟指数，并在运行最大值不需要时避免不必要的重新缩放，从而减轻较慢的非 矩阵乘法（Matmul） 资源的压力。 [7]

反向传播（Backward Pass） 有一个不同的瓶颈。 FA4 使用 Blackwell 的 张量（Tensor） 内存 (TMEM) 和 2-CTA 矩阵乘加（MMA） 模式来减少共享 内存流量（Memory Traffic） 和原子 累加（Accumulation） 开销。 [7]

这就产生了一个重要的系统原理：

内核（Kernel） 根据特定的硬件比率进行平衡。如果 张量核心吞吐量（Tensor-core Throughput） 的改进速度快于 内存带宽（Memory Bandwidth） 或特殊功能 吞吐量（Throughput），则即使数学函数相同，最佳算法 流水线（Pipeline） 也可能会发生变化。

这就是为什么“FlashAttention-4 只是更快的 FA3”错过了有趣的部分。该算法和 内核（Kernel） 流水线（Pipeline） 是围绕新的不对称性共同设计的。 FlashAttention-4 标题本身就明确表明了这一点。[7]

这也解释了为什么 性能（Performance） 编号应始终命名为 GPU 代际（GPU Generation）。对于 H100、A100 或非 NVIDIA 加速器上的相同比率，B200 上的加速不是 证据（Evidence）。

28 FA4 and asymmetric hardware scaling

The FlashAttention lineage is a case study in why kernels cannot be optimized once and assumed optimal forever.

On Blackwell, matrix multiplication became fast enough that the forward pass can become constrained by softmax/exponential work relative to tensor-core GEMM. FA4 uses software-emulated exponential and avoids unnecessary rescaling when the running maximum does not require it, reducing pressure on slower non-matmul resources.[7]

The backward pass has a different bottleneck. FA4 uses Blackwell's Tensor Memory (TMEM) and 2-CTA MMA mode to reduce shared-memory traffic and atomic accumulation overhead.[7]

This yields an important systems principle:

> **A kernel is balanced against a particular hardware ratio.** If tensor-core throughput improves faster than memory bandwidth or special-function throughput, the optimal algorithmic pipeline can change even when the mathematical function is identical.

That is why "FlashAttention-4 is just a faster FA3" misses the interesting part. The algorithm and the kernel pipeline are co-designed around new asymmetries. The FlashAttention-4 title itself makes that point explicit.[7]

This also explains why performance numbers should always name the GPU generation. A speedup on B200 is not evidence for the same ratio on H100, A100, or a non-NVIDIA accelerator.

29 FA4实施及现状

对应原文第 33 页；点击查看相邻原文对照页

29 FA4 实现和当前状态 当前官方 Dao-AILab 存储库将 FlashAttention-4 公开为优化的 CuTeDSL 实现

包。[8]

FlashAttention-4 论文强调了另一个工程变更：实现是用嵌入 Python 的 CuTe-DSL 编写的。该论文报告称，其编译时间比比较中使用的传统 C++ 模板方法快 20–30 倍，同时保留所需的低级表达能力。 [7]

将其视为当前实现快照，而不是常青属性。“FA4 存在于官方存储库中”和“每个生产环境都应该用 FA4 替换 FA2”是截然不同的说法。

硬件、CUDA、PyTorch、功能和封装要求快速发展。持久的教训是算法进展，而在部署之前应始终根据当前的官方存储库检查确切的安装/支持声明。

截至 2026 年 9 月，PyPI 将 flash-attn-4 分类为开发状态 :: 3 - Alpha。[9]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

29 FA4 implementation and current status

The current official Dao-AILab repository exposes FlashAttention-4 as a CuTeDSL implementation optimized for Hopper and Blackwell GPUs such as H100 and B200, with installation through the `flash-attn-4` package.[8]

The FlashAttention-4 paper emphasizes another engineering change: the implementation is written in CuTe-DSL embedded in Python. The paper reports 20–30× faster compile times than the traditional C++ template-based approach used in its comparison while retaining the required low-level expressivity.[7]

As of September 2026, PyPI classifies flash-attn-4 as

```
Development Status :: 3 - Alpha.[9]
```

> Treat this as a current implementation snapshot, not an evergreen property. “FA4 exists in the official repository” and “every production environment should replace FA2 with FA4” are very different claims.

Hardware, CUDA, PyTorch, feature, and packaging requirements evolve quickly. The durable lesson is the algorithmic progression, while exact installation/support claims should always be checked against the current official repository before deployment.

30 FlashAttention-1 到 -4 比较

对应原文第 34 页；点击查看相邻原文对照页

30 FlashAttention-1 到 -4 比较 主要目标 核心（Core） 内核（Kernel）/算法重点 FA1 IO / HBM 流量 分块（Tiling）、在线 Softmax（Online Softmax）、融合 精确注意力（Exact Attention）、IO 分析 NeurIPS 2022 论文报告了 训练（Training） 的显着加速与当时的基线相比。[4]

FA2 GPU 利用率（GPU Utilization） 更多的序列级并行性，更少的非 矩阵乘法（Matmul） 浮点运算次数（FLOPs），改进的 线程束（Warp） 工作划分（Work Partitioning） 大约是 FA1 的 2 倍，并且在论文中的 A100 上理论最大 浮点运算次数（FLOPs）/s 为 50–73%。 [5]

FA3 料斗 线程束（Warp） 专用，异步 TMA/WGMMA 流水线（Pipeline）、通用矩阵乘法（GEMM）-

FA4 Blackwell 时代的不对称 完全异步 矩阵乘加（MMA） 管道、更大的图块、软件 exp/条件重新缩放、TMEM 和 2-CTA 后向技术 B200 上高达 1613 TFLOPs/s BF16

版本号不应被解释为不同 注意力机制（Attention） 定义的序列。该系列保留了 高效注意力（Efficient Attention） 评估的中心目标，同时使调度和数值路径适应更新的硬件。

确切的功能 矩阵（Matrix） 因实现而异。例如，FA3 与 Hopper 特定的优化密切相关，而当前的 FA4 存储库通过其 CuTeDSL 路径同时针对 Hopper 和 Blackwell。 [8]

选定的论文报告结果是 H100 上 FA2 的 1.5-2.0 倍，BF16 高达 840 TFLOPs/s。[6]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

30 FlashAttention-1 through -4 compared

	Main target	Core kernel/algorithm emphasis	Selected paper-reported results
FA1	IO / HBM traffic	Tiling, online softmax, fused exact attention, IO analysis	NeurIPS 2022 paper reports substantial training speedups vs. then-current baselines.[4]
FA2	GPU utilization	More sequence-level parallelism, fewer non-matmul FLOPs, improved warp work partitioning	About 2× over FA1 and 50–73% theoretical max FLOPs/s on A100 in paper.[5]
FA3	Hopper	Warp specialization, asynchronous TMA/WGMMA pipeline, GEMM-softmax overlap, FP8 path	1.5-2.0x over FA2 on H100 in paper, BF16 up to 840 TFLOPs/s.[6]
FA4	Blackwell-era asymmetry	Fully async MMA pipelines, larger tiles, software exp / conditional rescale, TMEM and 2-CTA backward techniques	Up to 1613 TFLOPs/s BF16 on B200 and 1.3× vs. cuDNN 9.13 in paper.[7]

> The version number should not be interpreted as a sequence of different attention definitions. The family preserves the central goal of efficient attention evaluation while adapting scheduling and numerical paths to newer hardware.

The exact feature matrix differs across implementations. For example, FA3 is strongly associated with Hopper-specific optimization, while the current FA4 repository targets both Hopper and Blackwell through its CuTeDSL path.[8]

31 PyTorch 今天缩放-点积注意力（Dot-product Attention）

对应原文第 35 页；点击查看相邻原文对照页

31 PyTorch 已缩放-点积注意力（Dot-product Attention） 今天 高级 PyTorch API 是

查询（Query）、键（Key）、值（Value）、attn_掩码（Mask）=无、随机失活（Dropout）_p=0.0、is_causal=False、scale=无、enable_gqa=False、

当前的 PyTorch 文档称 CUDA SDPA 可以根据输入自动选择优化的实现。主要 API 文档明确讨论了 FlashAttention-2、内存 高效注意力（Efficient Attention） 实现和 C++ 数学实现。限制后端的首选机制是 torch.nn.注意力机制（Attention）.sdpa_内核（Kernel）.[10, 11]

例如：

与 sdpa_内核（Kernel）（SDPBackend.FLASH_注意力机制（Attention））：

y = F.scaled_dot_product_注意力机制（Attention）(q, k, v, is_causal=True) 如果请求的 融合内核（Fused Kernel） 无法针对提供的输入运行，当其他回退被禁用时，PyTorch 会发出警告并说明原因。后端资格取决于设备、数据类型、形状、掩码（Mask） 和其他参数。[10]

当前的 torch.nn.注意力机制（Attention） 文档还公开了用于较新的 FlashAttention 实现的注册/激活（Activation） 挂钩，包括 FA3 和 FA4 等标识符。这是一个快速移动的集成界面，因此请使用已安装的 PyTorch 版本的文档作为权威。[12]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

31 PyTorch scaled-dot-product attention today

The high-level PyTorch API is

```
torch.nn.functional.scaled_dot_product_attention(
    query, key, value,
    attn_mask=None,
    dropout_p=0.0,
    is_causal=False,
    scale=None,
    enable_gqa=False,
)
```

Current PyTorch documentation says CUDA SDPA can automatically select optimized implementations based on the inputs. The main API documentation explicitly discusses FlashAttention-2, a memory-efficient attention implementation, and the C++ math implementation. The preferred mechanism for restricting the backend is `torch.nn.attention.sdpa_kernel`.[10, 11]

For example:

```
from torch.nn.attention import SDPBackend, sdpa_kernel
import torch.nn.functional as F

with sdpa_kernel(SDPBackend.FLASH_ATTENTION):
    y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

If a requested fused kernel cannot run for the supplied inputs, PyTorch can warn with reasons when other fallbacks are disabled. Backend eligibility depends on device, dtype, shape, mask, and other arguments.[10]

Current `torch.nn.attention` documentation also exposes registration/activation hooks for newer FlashAttention implementations, including identifiers such as FA3 and FA4. This is a fast-moving integration surface, so use the installed PyTorch version's documentation as authority.[12]

32 精确性不是 逐位一致性（Bitwise Identity）

对应原文第 36 页；点击查看相邻原文对照页

32 准确性不是 逐位一致性（Bitwise Identity） 假设参考数学 内核（Kernel） 和 FlashAttention 内核（Kernel） 都实现相同的密集 SDPA 定义。

您仍然应该预料到较小的数值差异。

浮点（Floating-point） 加法不结合：

对于某些 值（Values），在有限的 精度（Precision） 中。 分块（Tiling） 更改缩减顺序。当 值（Values） 被舍入时，融合可能会改变。 累加（Accumulation） 精度（Precision） 可能有所不同。 PyTorch 的再现性文档明确指出，SDPA 后端可以产生不同的结果，因为它们以不同的顺序执行 浮点（Floating-point） 累加（Accumulation）。 [13]

因此，数值验证应使用适当的容差而不是按位相等。预期的容差取决于：

- 累加（Accumulation） 精度（Precision），

- 序列长度和分数分布，

- 硬件/后端，

- 随机失活（Dropout）/随机性。

精确的算法意味着没有故意对 稠密注意力（Dense Attention） 进行数学近似。

按位相同的实现意味着每个有限 精度（Precision） 操作都落在完全相同的位上。 FlashAttention 承诺第一个想法，而不是任意 内核（Kernels） 的第二个想法。

决定论是另一个单独的问题。当前的库公开了特定于后端和版本的确定性行为控制，这可能会带来 性能（Performance） 或内存成本。 [8, 13]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

32 Exactness is not bitwise identity

Suppose a reference math kernel and a FlashAttention kernel both implement the same dense SDPA definition. You should still expect small numerical differences.

Floating-point addition is not associative:

$$(a + b) + c \neq a + (b + c)$$

in finite precision for some values. Tiling changes reduction order. Fusion can change when values are rounded. Accumulation precision can differ. PyTorch's reproducibility documentation explicitly notes that SDPA backends can produce different results because they perform floating-point accumulation in different orders.[13]

Therefore numerical validation should use appropriate tolerances rather than bitwise equality. The expected tolerance depends on:

- FP32 vs. BF16/FP16/FP8,
- accumulation precision,
- sequence length and score distribution,
- hardware/backend,
- dropout/randomness.

> **Exact algorithm** means no deliberate mathematical approximation of dense attention.
> **Bitwise identical implementation** means every finite-precision operation lands on exactly the same bits. FlashAttention promises the first idea, not the second across arbitrary kernels.

Determinism is another separate question. Current libraries expose backend- and version-specific controls for deterministic behavior, which can carry performance or memory costs.[8, 13]

33 FlashAttention 与 PagedAttention

对应原文第 37 页：点击查看相邻原文对照页

33 FlashAttention 与 PagedAttention 名称听起来相关，但它们解决不同的问题。

FlashAttention PagedAttention 主要问题 高效执行 注意力机制（Attention），特别是减少大型中间 内存流量（Memory Traffic） 管理 LLM 动态增长的 KV 缓存内存 推理服务（Serving） 机制 分块（Tiling）、在线 Softmax（Online Softmax）、融合/硬件 内核（Kernels）受分页启发的 KV 块分配和映射 主状态 注意力机制（Attention） 切片和行 归一化（Normalization）/输出状态 持久每个请求 键值缓存（KV Cache） 更改 稠密注意力（Dense Attention） 公式?

密集 FlashAttention 否 否，主要更改缓存内存管理 PagedAttention 随 vLLM 一起引入，以减少 碎片化（Fragmentation） 中的 KV 缓存浪费以及高 吞吐量（Throughput） 推理服务（Serving） 中的重复。[14]

FlashAttention 是关于 注意力机制（Attention） 操作如何消耗 Q/K/V 的。 PagedAttention 是关于 推理服务（Serving） 系统如何跨请求布局和访问持久 KV 状态。它们可以共存于同一个 推理服务（Serving） 堆栈中。

当前的 Dao-AILab 存储库甚至公开了一个面向 KV 缓存的 FlashAttention 接口以及可选的块表，说明 内核（Kernel） 执行和缓存分页是可组合的问题，而不是相互排斥的想法。 [8]

如果问题陈述包含“N 2 分数/概率中间值”，请考虑 FlashAttention。如果它包含“碎片化的每个请求 KV 缓存分配”，请考虑 PagedAttention。

33 FlashAttention vs. PagedAttention

The names sound related, but they solve different problems.

	FlashAttention	PagedAttention
Primary problem	Efficiently execute attention, especially reducing large intermediate memory traffic	Manage dynamically growing KV-cache memory for LLM serving
Mechanism	Tiling, online softmax, fused/hardware-aware kernels	Paging-inspired KV-block allocation and mapping
Main state	Attention tiles and row-wise normalization/output state	Persistent per-request KV cache
Changes dense attention formula?	No for dense FlashAttention	No, primarily changes cache memory management

PagedAttention was introduced with vLLM to reduce KV-cache waste from fragmentation and duplication in high-throughput serving.[14]

FlashAttention is about how an attention operation consumes Q/K/V. PagedAttention is about how serving systems lay out and access persistent KV state across requests. They can coexist in the same serving stack.

The current Dao-AILab repository even exposes a KV-cache-oriented FlashAttention interface with optional block tables, illustrating that kernel execution and cache paging are composable concerns rather than mutually exclusive ideas.[8]

> If the problem statement contains "N^2 score/probability intermediates," think FlashAttention. If it contains "fragmented per-request KV-cache allocation," think PagedAttention.

34 FlashAttention 与稀疏和 线性注意力（Linear Attention）

对应原文第 38 页；点击查看相邻原文对照页

34 FlashAttention 与稀疏和 线性注意力（Linear Attention） 三种截然不同的策略通常组合在一起，因为在某些情况下所有策略都可以使 注意力机制（Attention） 更便宜。

Dense FlashAttention 保留密集的 Softmax 归一化（Softmax）-注意力机制（Attention） 函数并更改其执行方式。

Sparse/局部注意力（Local Attention） 故意仅计算 查询（Query）-键（Key） 交互的子集。如果每个 查询（Query） 仅关注一个窗口或选定的块，则算术可能会低于完全密集N 2 工作，但 模型（Model） 的 注意力机制（Attention） 模式已更改。

线性 注意力机制（Attention） 系列改变了数学公式，通常使用特征图或关联性，因此可以在没有显式密集 Softmax 归一化（Softmax） 分数矩阵（Score Matrix） 的情况下重新排列 注意力机制（Attention）。它们的语义和近似/精确性属性取决于具体的方法。

短语“memory-高效注意力（Efficient Attention）”过于宽泛，无法识别方法。问两个问题：

1. 它是否计算出相同密集的 Softmax 归一化（Softmax） 注意力机制（Attention）？

2.它是否减少了算术对，或者主要是内存流量（Memory Traffic）/中间存储?

最初的 FlashAttention 论文明确地将其密集精确算法与近似方法进行了对比，并单独探索了块稀疏 FlashAttention 作为近似/稀疏扩展。 [4]

这就是为什么“FlashAttention 使 注意力机制（Attention） 呈线性”是错误的。如果系统由于使用 局部窗口（Local Window） 或其他稀疏模式而显示近线性 缩放（Scaling），则稀疏性会改变交互数量。

FlashAttention 可能仍然是下面的 内核（Kernel）。

34 FlashAttention vs. sparse and linear attention

Three very different strategies are often grouped together because all can make attention cheaper in some regime.

Dense FlashAttention keeps the dense softmax-attention function and changes how it is executed.

Sparse/local attention deliberately computes only a subset of query-key interactions. If each query attends to only a window or selected blocks, arithmetic can fall below full dense N^2 work, but the model's attention pattern has changed.

Linear-attention families change the mathematical formulation, often using feature maps or associativity so attention can be rearranged without an explicit dense softmax score matrix. Their semantics and approximation/exactness properties depend on the specific method.

> The phrase "memory-efficient attention" is too broad to identify a method. Ask two questions:
> 1. Does it compute the same dense softmax attention?
> 2. Does it reduce arithmetic pairs, or mostly memory traffic/intermediate storage?

The original FlashAttention paper explicitly contrasted its dense exact algorithm with approximate methods, and separately explored block-sparse FlashAttention as an approximate/sparse extension.[4]

This is why "FlashAttention makes attention linear" is wrong. If a system shows near-linear scaling because it uses a local window or another sparse pattern, the sparsity is what changed the number of interactions. FlashAttention may still be the kernel underneath.

35 训练（Training）、预填充（Prefill） 和解码是不同的机制

对应原文第 39 页；点击查看相邻原文对照页

35 训练（Training）、预填充（Prefill） 和解码是不同的机制 当 查询（Query） 和 键（Key） 序列维度都很大时（如 训练（Training） 或长提示符 预填充（Prefill） 中），FlashAttention 自然是最强大的。有许多 查询（Query） 行，因此避免具体化 注意力机制（Attention） 矩阵（Matrix） 并利用平铺的 GEMM 会带来很大的好处。

自回归解码则不同。对于单个新的词元（Token），Nq可以是1，而Nk是累积的上下文长度。该操作读取一个大的 键值缓存（KV Cache），但每个序列仅生成一个新的 查询（Query） 行。

在这种情况下，KV 缓存带宽和 推理服务（Serving）/批处理（Batching） 行为可以占主导地位，并且专门解码 内核（Kernels） 很重要。官方存储库公开了 flash_attn_with_kvcache，它可以在一个 内核（Kernel） 中更新和处理缓存，并支持 MQA/GQA 和可选 RoPE 处理等功能。 [8]

这并不意味着 FlashAttention 与 推理（Inference） 无关。 预填充（Prefill） 是 注意力机制（Attention） -heavy 密集机制，解码仍然可以使用来自同一实现系列的专用 内核（Kernels）。这意味着“FlashAttention 加速LLM 推理（Inference）”是不完整的，除非你说出哪个阶段和瓶颈。

FlashAttention 也无法消除其他成本：MLP 层、张量（Tensor） 并行通信、KV-缓存容量（Cache Capacity） 、模型（Model） 权重带宽、采样和调度程序开销仍然是单独的问题。

35 Training, prefill, and decode are different regimes

FlashAttention is most naturally powerful when both query and key sequence dimensions are substantial, as in training or long-prompt prefill. There are many query rows, so avoiding the materialized attention matrix and exploiting tiled GEMMs provide large benefits.

Autoregressive decode is different. For a single new token, N_q may be one while N_k is the accumulated context length. The operation reads a large KV cache but produces only one new query row per sequence. In that regime, KV-cache bandwidth and serving/batching behavior can dominate, and specialized decode kernels matter. The official repository exposes `flash_attn_with_kvcache`, which can update and attend to the cache in one kernel and supports features such as MQA/GQA and optional RoPE handling.[8]

> This does not mean FlashAttention is irrelevant to inference. Prefill is an attention-heavy dense regime, and decode can still use specialized kernels from the same implementation family. It means "FlashAttention speeds up LLM inference" is incomplete unless you say which phase and bottleneck.

FlashAttention also cannot eliminate other costs: MLP layers, communication in tensor parallelism, KV-cache capacity, model-weight bandwidth, sampling, and scheduler overhead remain separate concerns.

36 种常见的实施错误

对应原文第 40 页；点击查看相邻原文对照页

36 种常见的实施错误

1. 假设闪存后端始终运行。当 dtype、device、shape 或 掩码（Mask） 不受支持时，高级框架可能会回退。重要时使用后端控件/性能剖析（Profiling）。[10]

2. 在评估期间保留 随机失活（Dropout）。PyTorch SDPA 完全按照提供的方式应用 dropout_p，因此当评估不需要 随机失活（Dropout） 时传递 0.0。[10]

3. 使用错误的 掩码（Mask） 约定。布尔掩码（Boolean Mask） 语义因 API 而异。

4. 期望按位一致。融合内核（Fused Kernels） 可以重新排序 浮点（Floating-point） 操作。[13]

5. 忽略GQA头约束。当前接口需要兼容/可分割的查询（Query）和KV头计数。[8, 10]

6. 将当前的头维度/数据类型支持视为永恒的。这些约束随着版本和后端的不同而变化。[8]

7. 没有 同步（Synchronization）/warmup 的基准测试。异步 GPU 执行可能会使简单的实际运行测量变得毫无意义。

8. 将稀疏/局部模式称为“密集 FlashAttention”。内核（Kernel） 和 注意力机制（Attention） 模式是单独的选择。

9. 将预填充（Prefill）与解码混淆。形状和瓶颈不同。

10. 假设最新的 代际（Generation） 始终可部署。当前 FA4 包元数据为 alpha，必须检查硬件/软件兼容性。[9]

36 Common implementation mistakes

1. **Assuming the flash backend always ran.** High-level frameworks may fall back when dtype, device, shape, or mask is unsupported. Use backend controls/profiling when it matters.[10]
2. **Leaving dropout on during evaluation.** PyTorch SDPA applies `dropout_p` exactly as supplied, so pass 0.0 when evaluation requires no dropout.[10]
3. **Using the wrong mask convention.** Boolean mask semantics differ across APIs.
4. **Expecting bitwise agreement.** Fused kernels can reorder floating-point operations.[13]
5. **Ignoring GQA head constraints.** Current interfaces require compatible/divisible query and KV head counts.[8, 10]
6. **Treating current head-dimension/dtype support as timeless.** These constraints change across releases and backends.[8]
7. **Benchmarking without synchronization/warmup.** Asynchronous GPU execution can make naïve wall-clock measurements meaningless.
8. **Calling a sparse/local pattern "dense FlashAttention."** Kernel and attention pattern are separate choices.
9. **Confusing prefill with decode.** The shapes and bottlenecks are different.
10. **Assuming newest generation is always deployable.** Current FA4 package metadata is alpha, and hardware/software compatibility must be checked.[9]

37 常见误解和实用 理解框架（Mental Model）

对应原文第 41 页；点击查看相邻原文对照页

37 常见误解和实用 理解框架（Mental Model）

Dense FlashAttention 使用在线平铺 归一化（Normalization） 评估密集 Softmax 归一化（Softmax）-注意力机制（Attention） 函数。[4]

“FlashAttention 引起关注 O(N)。”不会。稠密算术仍然是O(N 2d)。大的辅助注意力机制（Attention）-内存占用（Memory Footprint）在序列长度上变得线性，并且HBM流量减少。

“这只是内核（Kernel）融合。”融合是实现故事的一部分，但在线 Softmax 归一化（Softmax） 分块（Tiling） 和 IO 感知调度是算法性的，而不仅仅是连接现有的 内核（Kernels）。

“与PagedAttention相同。”否。PagedAttention 管理 推理服务（Serving） 的 KV 缓存分配。[14]

“精确意味着按位相同。” No. 浮点（Floating-point） 操作顺序可以更改舍入（Rounding）。[13]

“长上下文变得自由。”不会。密集成对算术仍然是二次的，并且 KV 缓存/其他 模型（Model） 成本仍然存在。

“FA1、FA2、FA3 和 FA4 是不同的 注意力机制（Attention） 架构。”不。它们是由不同硬件瓶颈塑造的连续几代高效 内核（Kernels）/算法。[4,

实用的 理解框架（Mental Model）:FlashAttention 将分数图块保持在靠近计算单元的位置，仅携带精确合并 Softmax 归一化（Softmax） 块所需的行统计信息，将每个概率图块立即消耗到 V 累加（Accumulation） 中，并避免通过 HBM 往返 fullN 2 注意力机制（Attention） 矩阵（Matrix）。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

37 Common misconceptions and the practical mental model

“FlashAttention approximates softmax.”
No. Dense FlashAttention evaluates the dense softmax-attention function using tiled online normalization.[4]

“FlashAttention makes attention $O(N)$.”
No. Dense arithmetic remains $O(N^2d)$. The large auxiliary attention-memory footprint becomes linear in sequence length, and HBM traffic is reduced.

“It is just kernel fusion.”
Fusion is part of the implementation story, but the online-softmax tiling and IO-aware scheduling are algorithmic, not merely concatenating existing kernels.

“It is the same as PagedAttention.”
No. PagedAttention manages KV-cache allocation for serving.[14]

“Exact means bitwise identical.”
No. Floating-point operation order can change rounding.[13]

“Long context becomes free.”
No. Dense pairwise arithmetic is still quadratic, and KV-cache / other model costs remain.

“FA1, FA2, FA3, and FA4 are different attention architectures.”
No. They are successive generations of efficient kernels/algorithms shaped by different hardware bottlenecks.[4, 5, 6, 7]

Practical mental model: FlashAttention keeps score tiles close to the compute units, carries only the row statistics needed to merge softmax blocks exactly, consumes each probability tile immediately into the V accumulation, and avoids round-tripping a full N^2 attention matrix through HBM.

参考文献和进一步阅读

对应原文第 42 页；点击查看相邻原文对照页

参考文献和进一步阅读

参考文献

[1] Vaswani, A. 等人。注意力机制（Attention） 就是您所需要的。 NeurIPS 2017。 https://arxiv.org/abs/1706.03762

[2] Milakov, M., Gimelshein, N.Softmax 归一化（Softmax） 的在线标准化器计算。 arXiv：1805.02867，2018。https://arxiv。

组织/abs/1805.02867

[3] Rabe, M. N., Staats, C.自注意力（Self-attention） 不需要 O(n2) 内存。 arXiv：2112.05682，2021/2022。https：

//arxiv.org/abs/2112.05682

[4] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C.FlashAttention：具有 IO 感知功能的快速且内存高效的 精确注意力（Exact Attention）。 NeurIPS 2022.https://arxiv.org/abs/2205.14135

[5] Dao，T.FlashAttention-2：更快的 注意力机制（Attention），具有更好的并行性和 工作划分（Work Partitioning）。 ICLR 2024.https：

//arxiv.org/abs/2307.08691

[7] Zadouri, T.、Hoehnerbach, M.、Shah, J.、Liu, T.、Thakkar, V.、Dao, T.FlashAttention-4：非对称硬件 缩放（Scaling） 的算法和 内核（Kernel） 流水线协同设计。 arXiv：2603.05451，2026。 https://arxiv.org/abs/2603。

[8]道-AILab。 flash-注意力机制（Attention） 官方存储库和当前实施文档。 https://github.com/Dao-AILab/flash-注意力机制（Attention）

[10] PyTorch.torch.nn.function.scaled_dot_product_注意力机制（Attention） 文档。 https://docs.pytorch.org/docs/stable/ generated/torch.nn.function.scaled_dot_product_注意力机制（Attention）.html

[11] PyTorch.torch.nn.注意力机制（Attention）.sdpa_内核（Kernel） 文档。 https://docs.pytorch.org/docs/stable/ generated/

[12] 火炬。 torch.nn.注意力机制（Attention） 模块和 FlashAttention 实现注册 API。 https://docs.

pytorch.org/docs/stable/nn.注意力机制（Attention）.html

[13] 火炬。再现性文档，包括缩放的 点积注意力（Dot-product Attention） 后端差异。 https：

//docs.pytorch.org/docs/stable/notes/randomness.html

[14] Kwon, W. 等人。大语言模型（Large Language Model） 推理服务（Serving） 与 PagedAttention 的高效内存管理。 SOSP

2023。https://arxiv.org/abs/2309.06180

[6] Shah, J.、Bikshandi, G.、Zhang, Y.、Thakkar, V.、Ramani, P.、Dao, T. FlashAttention-3：具有异步性和低 精度（Precision） 的快速准确的 注意力机制（Attention）。 NeurIPS 2024。DOI：10.52202/079017-2193。

[9]PyPI。 flash-attn-4 包元数据和发布历史记录；包分类器“开发状态::3 - Alpha”。https://pypi.org/project/flash-attn-4/

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

References and further reading

References

[1] Vaswani, A. et al. *Attention Is All You Need*. NeurIPS 2017. https://arxiv.org/abs/1706.03762

[2] Milakov, M., Gimelshein, N. *Online normalizer calculation for softmax*. arXiv:1805.02867, 2018. https://arxiv.org/abs/1805.02867

[3] Rabe, M. N., Staats, C. *Self-attention Does Not Need $O(n^2)$ Memory*. arXiv:2112.05682, 2021/2022. https://arxiv.org/abs/2112.05682

[4] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. https://arxiv.org/abs/2205.14135

[5] Dao, T. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR 2024. https://arxiv.org/abs/2307.08691

[6] Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., Dao, T. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*. NeurIPS 2024. DOI: 10.52202/079017-2193.

[7] Zadouri, T., Hoehnerbach, M., Shah, J., Liu, T., Thakkar, V., Dao, T. *FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling*. arXiv:2603.05451, 2026. https://arxiv.org/abs/2603.05451

[8] Dao-AILab. *flash-attention official repository and current implementation documentation*. https://github.com/Dao-AILab/flash-attention

[9] PyPI. flash-attn-4 package metadata and release history; package classifier "Development Status :: 3 - Alpha." https://pypi.org/project/flash-attn-4/

[10] PyTorch. *torch.nn.functional.scaled_dot_product_attention documentation*. https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html

[11] PyTorch. *torch.nn.attention.sdpa_kernel documentation*. https://docs.pytorch.org/docs/stable/generated/torch.nn.attention.sdpa_kernel.html

[12] PyTorch. *torch.nn.attention module and FlashAttention implementation registration APIs*. https://docs.pytorch.org/docs/stable/nn.attention.html

[13] PyTorch. *Reproducibility documentation, including scaled-dot-product attention backend differences*. https://docs.pytorch.org/docs/stable/notes/randomness.html

[14] Kwon, W. et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. https://arxiv.org/abs/2309.06180

参考图：针对哪个问题阅读什么内容

对应原文第 43 页；点击查看相邻原文对照页

参考图：针对哪个问题阅读什么内容 问题 最佳起始来源 原始 IO 感知算法是什么？

FlashAttention-1 论文 [4] 为什么在线 归一化（Normalization） 有效?

Milakov & Gimelshein [2] 精确的次二次记忆 注意力机制（Attention） 是否更早已知?

Rabe & Staats [3] 具体改变了什么

FlashAttention-2论文[5]Hopper具体是什么

FlashAttention-3 论文 [6] Blackwell 发生了什么变化

FlashAttention-4论文[7]当前官方包支持什么？

Dao-AILab 存储库 [8] 当前 PyTorch 如何选择 SDPA 内核（Kernels）？

PyTorch SDPA 和sdpa_内核（Kernel） 文档 [10, 11] 为什么确切的后端在数值上会有所不同?

PyTorch 重现性说明 [13] PagedAttention 有何不同？

权等人。 [14]

实施支持的变化速度比数学还要快。对于已部署的系统，将已安装框架的文档和确切的 内核（Kernel） 存储库/版本视为受支持的设备、dtypes、头维度（Head Dimensions）、掩码、随机失活（Dropout） 和 GQA 行为的权威。

面向人工智能工程师、开发人员和认真学习者的简短技术手册

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

Reference map: what to read for which question

Question	Best starting source
What is the original IO-aware algorithm?	FlashAttention-1 paper [4]
Why does online normalization work?	Milakov & Gimelshein [2]
Was exact subquadratic-memory attention known earlier?	Rabe & Staats [3]
What specifically changed in FA2?	FlashAttention-2 paper [5]
What is Hopper-specific in FA3?	FlashAttention-3 paper [6]
What changes on Blackwell in FA4?	FlashAttention-4 paper [7]
What does the current official package support?	Dao-AILab repository [8]
How does current PyTorch select SDPA kernels?	PyTorch SDPA and `sdpa_kernel` docs [10, 11]
Why can exact backends differ numerically?	PyTorch reproducibility notes [13]
How is PagedAttention different?	Kwon et al. [14]

Implementation support changes faster than the mathematics. For deployed systems, treat the installed framework's documentation and the exact kernel repository/release as authoritative for supported devices, dtypes, head dimensions, masks, dropout, and GQA behavior.